EDA

In [167…

```
%pip install mlxtend
%pip install skglm
```

```
Requirement already satisfied: mlxtend in /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.1
4/site-packages (0.24.0)
Requirement already satisfied: scipy>=1.16.3 in /Library/Frameworks/Python.framework/Versions/3.14/lib/pyth
on3.14/site-packages (from mlxtend) (1.17.0)
Requirement already satisfied: numpy>=2.3.5 in /Library/Frameworks/Python.framework/Versions/3.14/lib/pytho
n3.14/site-packages (from mlxtend) (2.4.2)
Requirement already satisfied: pandas>=2.3.3 in /Library/Frameworks/Python.framework/Versions/3.14/lib/pyth
on3.14/site-packages (from mlxtend) (3.0.0)
Requirement already satisfied: scikit-learn>=1.8.0 in /Library/Frameworks/Python.framework/Versions/3.14/li
b/python3.14/site-packages (from mlxtend) (1.8.0)
Requirement already satisfied: matplotlib>=3.10.8 in /Library/Frameworks/Python.framework/Versions/3.14/li
b/python3.14/site-packages (from mlxtend) (3.10.8)
Requirement already satisfied: joblib>=1.5.2 in /Library/Frameworks/Python.framework/Versions/3.14/lib/pyth
on3.14/site-packages (from mlxtend) (1.5.3)
Requirement already satisfied: contourpy>=1.0.1 in /Library/Frameworks/Python.framework/Versions/3.14/lib/p
ython3.14/site-packages (from matplotlib>=3.10.8->mlxtend) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /Library/Frameworks/Python.framework/Versions/3.14/lib/pytho
n3.14/site-packages (from matplotlib>=3.10.8->mlxtend) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /Library/Frameworks/Python.framework/Versions/3.14/lib/
python3.14/site-packages (from matplotlib>=3.10.8->mlxtend) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /Library/Frameworks/Python.framework/Versions/3.14/lib/
python3.14/site-packages (from matplotlib>=3.10.8->mlxtend) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /Library/Frameworks/Python.framework/Versions/3.14/lib/py
thon3.14/site-packages (from matplotlib>=3.10.8->mlxtend) (26.0)
Requirement already satisfied: pillow>=8 in /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.
14/site-packages (from matplotlib>=3.10.8->mlxtend) (12.1.1)
Requirement already satisfied: pyparsing>=3 in /Library/Frameworks/Python.framework/Versions/3.14/lib/pytho
n3.14/site-packages (from matplotlib>=3.10.8->mlxtend) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /Library/Frameworks/Python.framework/Versions/3.14/l
ib/python3.14/site-packages (from matplotlib>=3.10.8->mlxtend) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.1
4/site-packages (from python-dateutil>=2.7->matplotlib>=3.10.8->mlxtend) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.2.0 in /Library/Frameworks/Python.framework/Versions/3.14/l
ib/python3.14/site-packages (from scikit-learn>=1.8.0->mlxtend) (3.6.0)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: skglm in /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/s
ite-packages (0.5)
Requirement already satisfied: numpy>=1.12 in /Library/Frameworks/Python.framework/Versions/3.14/lib/python
3.14/site-packages (from skglm) (2.4.2)
Requirement already satisfied: numba in /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/s
ite-packages (from skglm) (0.64.0)
Requirement already satisfied: scikit-learn>=1.6 in /Library/Frameworks/Python.framework/Versions/3.14/lib/
```

```
python3.14/site-packages (from skglm) (1.8.0)
Requirement already satisfied: scipy>=0.18.0 in /Library/Frameworks/Python.framework/Versions/3.14/lib/pyth
on3.14/site-packages (from skglm) (1.17.0)
Requirement already satisfied: joblib>=1.3.0 in /Library/Frameworks/Python.framework/Versions/3.14/lib/pyth
on3.14/site-packages (from scikit-learn>=1.6->skglm) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in /Library/Frameworks/Python.framework/Versions/3.14/l
ib/python3.14/site-packages (from scikit-learn>=1.6->skglm) (3.6.0)
Requirement already satisfied: llvmlite<0.47,>=0.46.0dev0 in /Library/Frameworks/Python.framework/Versions/
3.14/lib/python3.14/site-packages (from numba->skglm) (0.46.0)
Note: you may need to restart the kernel to use updated packages.
```

In [168…

```python
# import libraries
import pandas as pd
import numpy as np
import scipy.stats as stats
import seaborn as sns
from matplotlib import pyplot as plt
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor as vif
from statsmodels.graphics.factorplots import interaction_plot
from sklearn.linear_model import LinearRegression

from mlxtend.feature_selection import ExhaustiveFeatureSelector as EFS
from mlxtend.feature_selection import SequentialFeatureSelector as SFS
from matplotlib.colors import LinearSegmentedColormap

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from skglm import GroupLasso
from sklearn.model_selection import GridSearchCV
from statsmodels.stats.anova import anova_lm
```

In [169…

```python
# load in dataset
basketball = pd.read_csv("cbb25.csv")
basketball.head()
```

Out[169…

	RK	Team	CONF	G	W	ADJOE	ADJDE	BARTHAG	EFG_O	EFG_D	…	FTRD	2P_O	2P_D	3P_O	3P_D	3PR
0	1	Houston	B12	34	30	124.8	88.0	0.9823	52.7	44.9	…	34.1	49.0	43.9	39.8	30.9	34.5
1	2	Duke	ACC	34	31	128.5	91.3	0.9807	57.4	44.5	…	25.4	58.0	43.4	37.7	30.9	45.4
2	3	Auburn	SEC	33	28	129.0	93.7	0.9756	55.7	46.0	…	39.2	56.1	47.2	36.8	29.2	40.6
3	4	Florida	SEC	34	30	127.7	94.0	0.9713	55.0	45.3	…	33.0	56.4	45.9	35.5	29.6	43.6
4	5	Alabama	SEC	33	25	127.6	96.4	0.9621	56.3	47.9	…	33.9	59.7	48.8	35.0	30.8	46.2

5 rows × 25 columns

Loading in the data set

In [170…

```
# only include needed variables
basketball = basketball[['W', 'CONF', 'ADJOE', 'ADJDE', 'ADJ_T']]
basketball.head()
```

Out[170…

	W	CONF	ADJOE	ADJDE	ADJ_T
0	30	B12	124.8	88.0	61.4
1	31	ACC	128.5	91.3	65.7
2	28	SEC	129.0	93.7	67.8
3	30	SEC	127.7	94.0	69.5
4	25	SEC	127.6	96.4	74.6

We wanted to use these varibles to see if they were a good predictor of wins. We didn't include any of the other varibles on the data set because we wanted to see if there was a distict correlation with these 3 varibles and conference to predict wins.

In [171…

```
# print info and null count
basketball.info()
basketball.isnull().sum()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 364 entries, 0 to 363
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   W       364 non-null    int64
 1   CONF    364 non-null    str
 2   ADJOE   364 non-null    float64
 3   ADJDE   364 non-null    float64
 4   ADJ_T   364 non-null    float64
dtypes: float64(3), int64(1), str(1)
memory usage: 15.5 KB
```

Out[171…

```
W        0
CONF     0
ADJOE    0
ADJDE    0
ADJ_T    0
dtype: int64
```

We just check to see if there were any null data points and there aren't any.

In [172…

```
# summary stats table
basketball.describe()
```

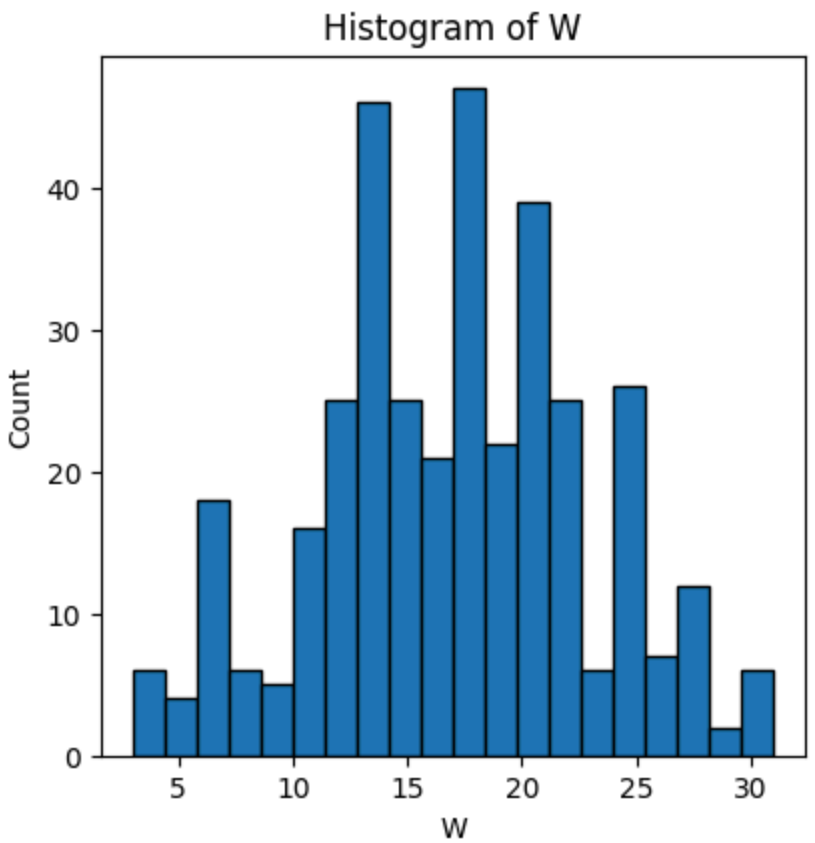
Out[172…

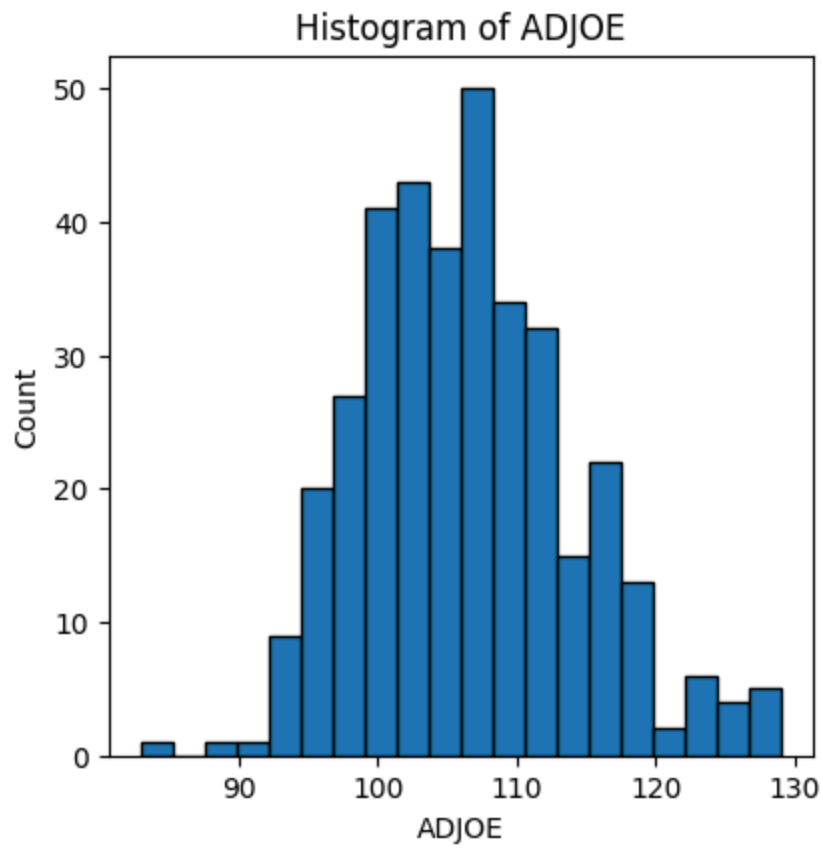
	W	ADJOE	ADJDE	ADJ_T
count	364.000000	364.000000	364.000000	364.000000
mean	16.934066	106.433516	106.433242	67.193681
std	5.872009	7.661871	6.620135	2.445514
min	3.000000	82.900000	88.000000	58.600000
25%	13.000000	100.875000	101.950000	65.575000
50%	17.000000	106.000000	106.750000	67.200000
75%	21.000000	111.275000	111.225000	68.800000
max	31.000000	129.000000	125.200000	74.600000

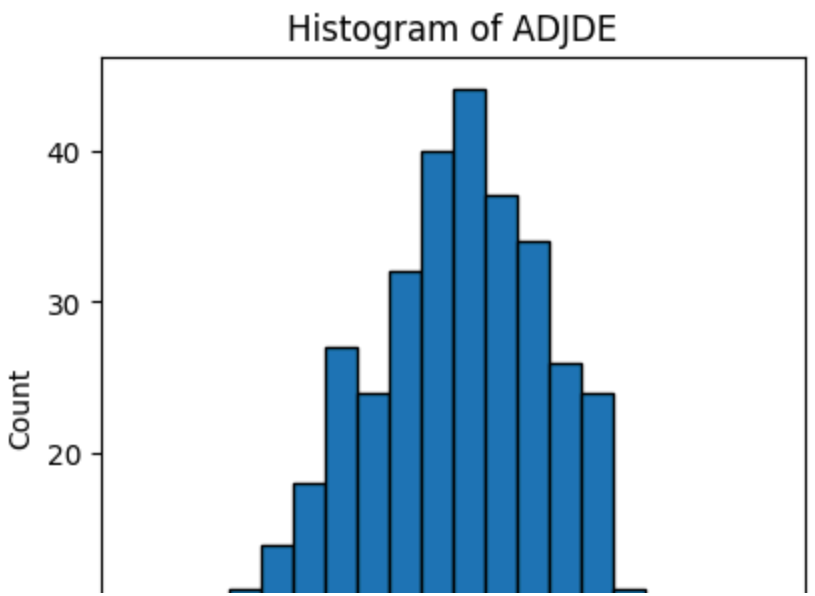
The Numeric columns look great nothing looks too out of the norm. showing us all our key statistics.

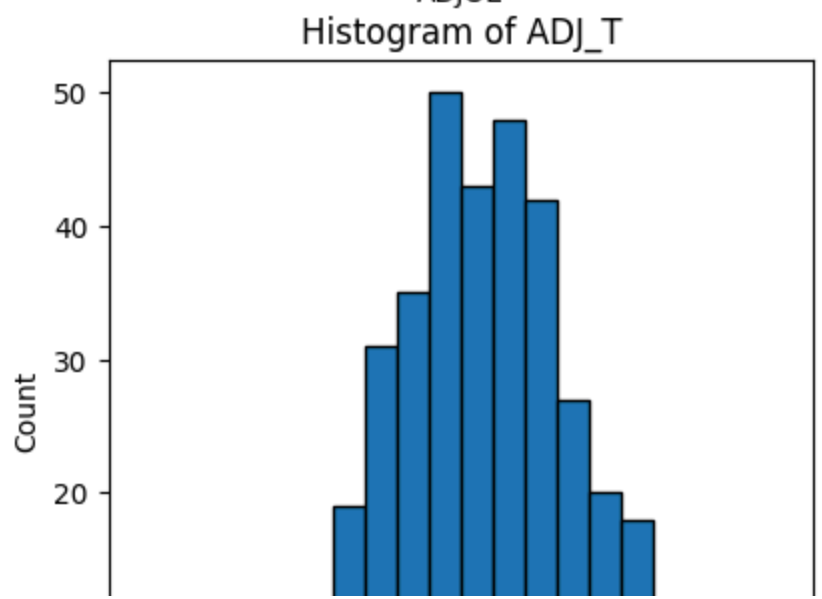
In [173…

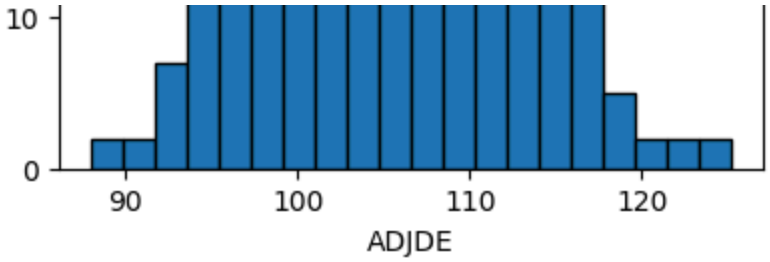
```
# Histograms of continous variables

continuous_cols = ["W", "ADJOE", "ADJDE", "ADJ_T"]

fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10, 10))

axes = axes.flatten()

for i, col in enumerate(continuous_cols):
    axes[i].hist(basketball[col].dropna(), bins=20, edgecolor='black')
    axes[i].set_title(f"Histogram of {col}")
    axes[i].set_xlabel(col)
    axes[i].set_ylabel("Count")

plt.show()
```

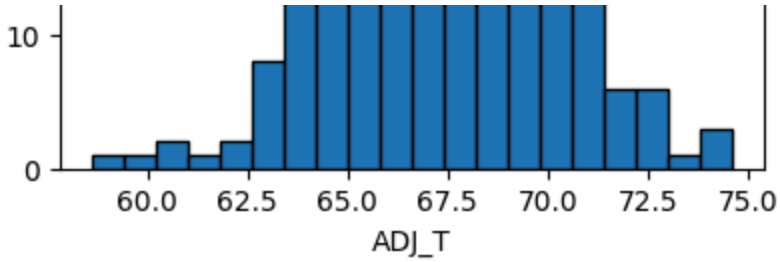

Histogram of W
Count
W
0
10
20
30
40
5
10
15
20
25
30


Histogram of ADJOE
Count
ADJOE
0
10
20
30
40
50
90
100
110
120
130


Histogram of ADJDE
Count
20
30
40


Histogram of ADJ_T
Count
20
30
40
50





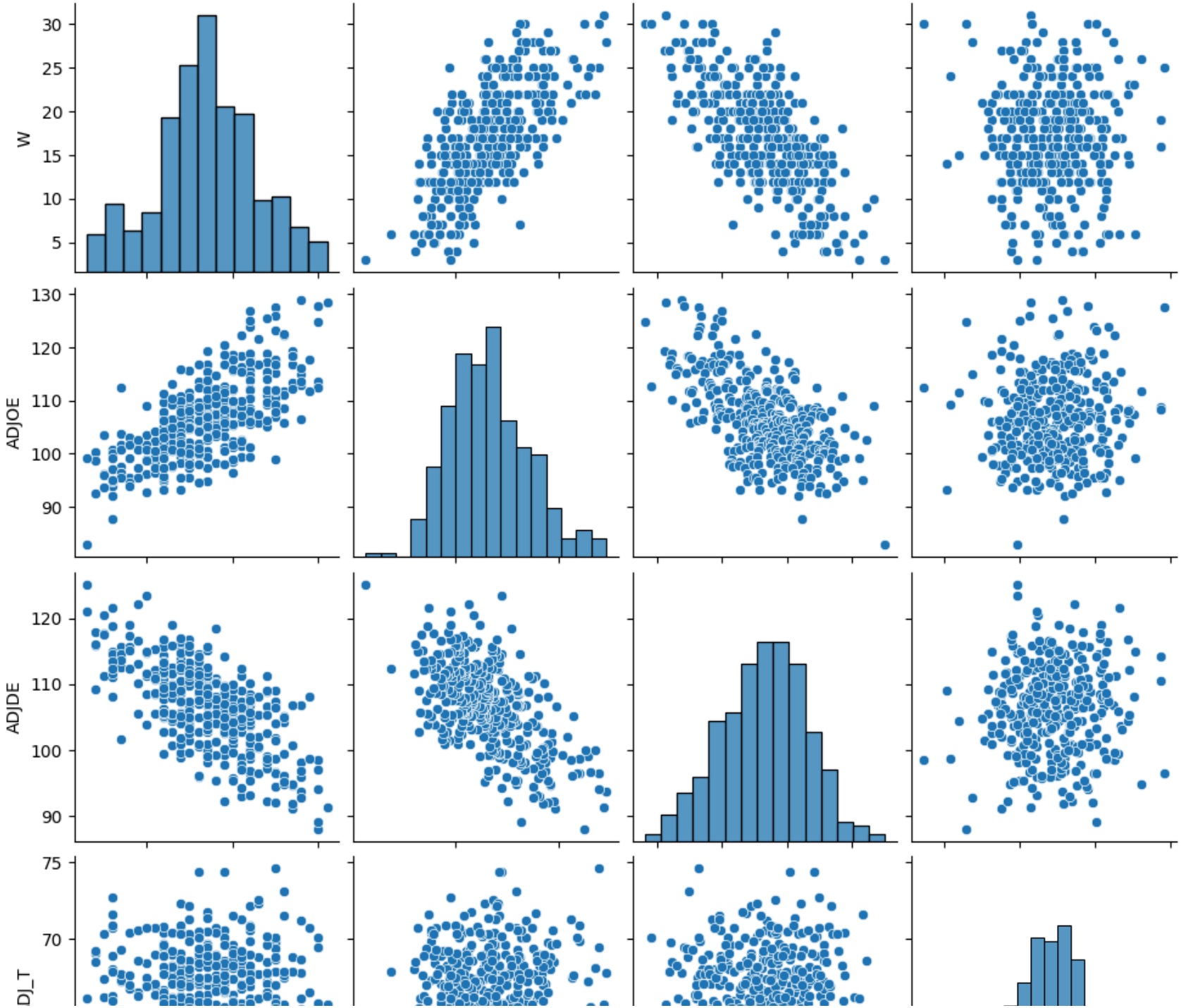
The histograms look really good. Nothing too skewed. we are also not seeing too many if any outliers on the histogram charts.

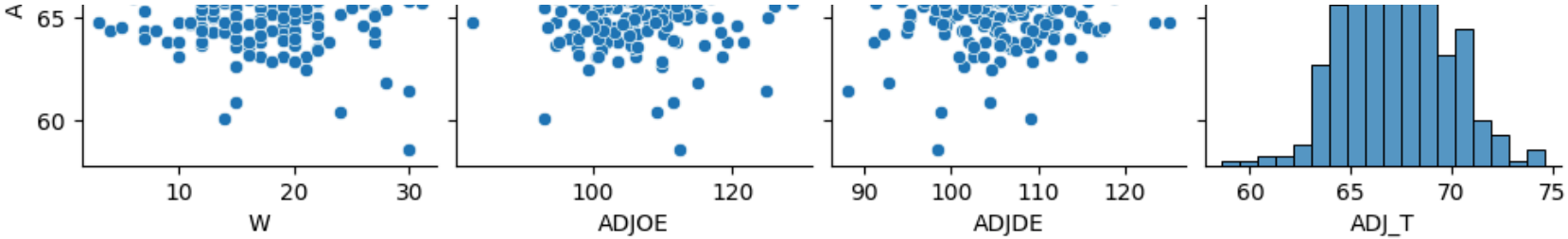
In [174…

```
# Scatterplot matrix of continous variables
sns.pairplot(basketball)
```

Out[174…

```
<seaborn.axisgrid.PairGrid at 0x12be5e650>
```

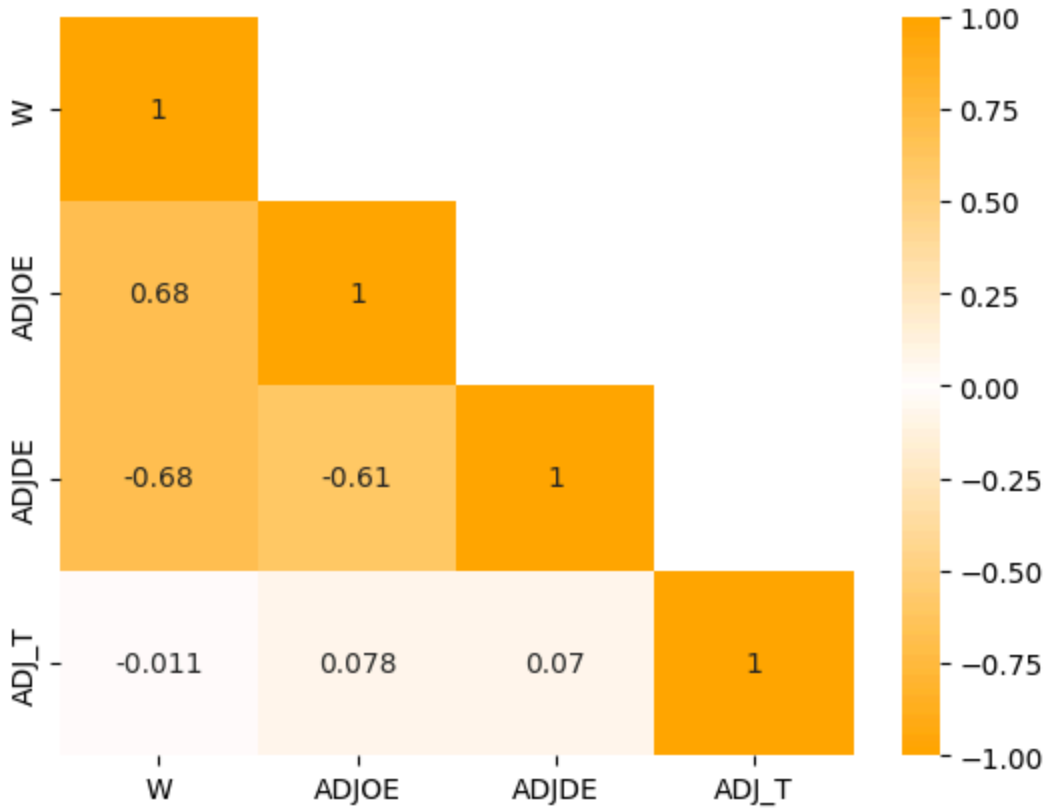

W
ADJOE
ADJDE
DJ_T
30
25
20
15
10
5
130
120
110
100
90
120
110
100
90
75
70



Here we are looking at the trends of the scatterplot. they all look really good. compared to wins, both ADJOE is strong positive linear trends and then ADJDE is a strong negative linear trend. And then ADJ_T doesn't look to have any trend. It looks like a clouded blob very dispersed. There doesn't seem to be multicolinearity in in the charts they all seem to have an effect of the graph in different ways.

In [175…

```
from matplotlib.colors import LinearSegmentedColormap
continuous_vars = ["W", "ADJOE", "ADJDE", "ADJ_T"]
# Custom symmetric colormap
cmap = LinearSegmentedColormap.from_list("bwb", ["orange", "white", "orange"])
corr = basketball[continuous_vars].corr()
sns.heatmap(
    corr,
    annot=True,
    vmin=-1, vmax=1,
    cmap=cmap,
    mask=np.triu(np.ones_like(corr), k=1))
plt.show()
```



ADJDE and ADJOE have a fairly strong correlation with each other being at -0.61 this could prove to have multicollinearity which could be bad for our model. ADJ_T doesn't really have any correlation with anything which is great. ADJOE and ADJDE both have a fairly strong correlation with Wins being at 0.68 and -0.68.

In [176…

```
# Number of rows in each comference
basketball["CONF"].value_counts()
```

Out[176…

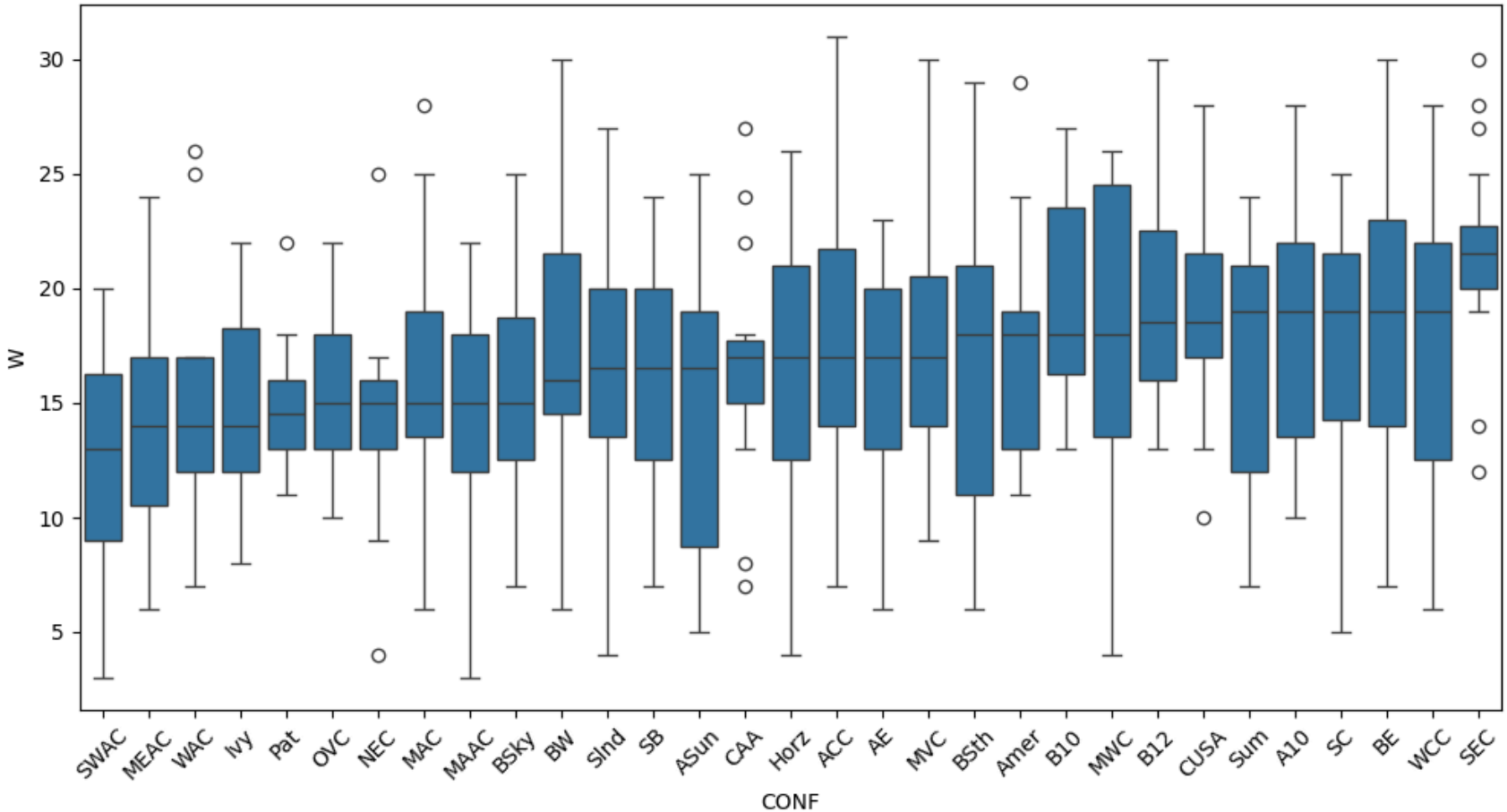
```
CONF
ACC     18
B10     18
B12     16
SEC     16
A10     15
SB      14
CAA     14
Amer    13
MAAC    13
MVC     12
Slnd    12
MAC     12
ASun    12
SWAC    12
WCC     11
BE      11
MWC     11
BW      11
Horz    11
OVC     11
CUSA    10
BSky    10
SC      10
Pat     10
WAC      9
BSth     9
Sum      9
AE       9
NEC      9
Ivy      8
MEAC     8
Name: count, dtype: int64
```

This just shows how many teams are in each conference. They all have different teams in the conferences. Not one team can be in multiple conferences. I would say there is no sparcity in the number of teams in each conference they seem to have reasonable amounts of teams in each conference.

In [177…

```
# Boxplots of categorical variables against the response variable
order = basketball.groupby("CONF")["W"].median().sort_values().index
```

```
plt.figure(figsize=(12,6))
sns.boxplot(x="CONF", y="W", data=basketball, order=order)
plt.xticks(rotation=45)
plt.show()
```



The boxplot shows each conference and the wins that they have. With the dots being an outlier. it looks like the majority of conferences average out to be around 15 - 17 wins. The SEC is higher on average with around 22 wins on average. The SWAC is also pretty low around 13 wins.

In [178…

```
basket = pd.get_dummies(basketball,
                        columns = ['CONF'],
                        dtype = int,
```

```
                                    drop_first = True)
basket.head()
```

Out[178…

	W	ADJOE	ADJDE	ADJ_T	CONF_ACC	CONF_AE	CONF_ASun	CONF_Amer	CONF_B10	CONF_B12	…	CONF_OVC
0	30	124.8	88.0	61.4	0	0	0	0	0	1	…	0
1	31	128.5	91.3	65.7	1	0	0	0	0	0	…	0
2	28	129.0	93.7	67.8	0	0	0	0	0	0	…	0
3	30	127.7	94.0	69.5	0	0	0	0	0	0	…	0
4	25	127.6	96.4	74.6	0	0	0	0	0	0	…	0

5 rows × 34 columns

We are creating a Dummy Varible for the conference so we can apply it to the OLS regression.

In [179…

```
y = basket['W']
X = sm.add_constant(basket.iloc[:, 1:])  # this is equivalent to above
mod = sm.OLS(y, X)
res = mod.fit()

basketball['residuals'] = res.resid
basketball['fittedvalues'] = res.fittedvalues

res.summary()
```

Out[179…

OLS Regression Results

Dep. Variable:	W	**R-squared:**	0.871
Model:	OLS	**Adj. R-squared:**	0.859
Method:	Least Squares	**F-statistic:**	67.79
Date:	Wed, 25 Mar 2026	**Prob (F-statistic):**	1.15e-126
Time:	12:22:32	**Log-Likelihood:**	-786.99
No. Observations:	364	**AIC:**	1642.
Df Residuals:	330	**BIC:**	1774.
Df Model:	33		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	20.2223	4.966	4.072	0.000	10.454	29.991
ADJOE	0.6465	0.024	27.258	0.000	0.600	0.693
ADJDE	-0.7174	0.027	-26.986	0.000	-0.770	-0.665
ADJ_T	0.0441	0.050	0.887	0.376	-0.054	0.142
CONF_ACC	-4.9622	0.780	-6.361	0.000	-6.497	-3.428
CONF_AE	7.1876	0.960	7.490	0.000	5.300	9.075
CONF_ASun	4.4574	0.881	5.059	0.000	2.724	6.191
CONF_Amer	1.1005	0.841	1.309	0.191	-0.553	2.754
CONF_B10	-7.7019	0.800	-9.623	0.000	-9.276	-6.127
CONF_B12	-9.2523	0.826	-11.205	0.000	-10.877	-7.628
CONF_BE	-5.8601	0.887	-6.606	0.000	-7.605	-4.115
CONF_BSky	4.0140	0.916	4.380	0.000	2.211	5.817
CONF_BSth	4.7811	0.949	5.037	0.000	2.914	6.648

CONF_BW	3.3476	0.882	3.794	0.000	1.612	5.083
CONF_CAA	4.4155	0.834	5.292	0.000	2.774	6.057
CONF_CUSA	0.8990	0.902	0.996	0.320	-0.876	2.674
CONF_Horz	4.7996	0.894	5.367	0.000	3.040	6.559
CONF_Ivy	1.3827	0.977	1.416	0.158	-0.538	3.304
CONF_MAAC	6.1644	0.865	7.125	0.000	4.462	7.866
CONF_MAC	4.8296	0.875	5.517	0.000	3.107	6.552
CONF_MEAC	7.7379	1.009	7.670	0.000	5.753	9.723
CONF_MVC	1.7910	0.859	2.085	0.038	0.101	3.481
CONF_MWC	-2.9811	0.878	-3.396	0.001	-4.708	-1.254
CONF_NEC	8.0483	0.973	8.268	0.000	6.133	9.963
CONF_OVC	7.0348	0.905	7.773	0.000	5.254	8.815
CONF_Pat	6.7246	0.932	7.212	0.000	4.890	8.559
CONF_SB	4.0366	0.833	4.843	0.000	2.397	5.676
CONF_SC	5.3014	0.916	5.786	0.000	3.499	7.104
CONF_SEC	-8.9653	0.844	-10.622	0.000	-10.626	-7.305
CONF_SWAC	7.6260	0.913	8.356	0.000	5.831	9.421
CONF_SInd	3.5489	0.867	4.092	0.000	1.843	5.255
CONF_Sum	2.6795	0.949	2.825	0.005	0.814	4.546
CONF_WAC	2.0133	0.941	2.139	0.033	0.161	3.865
CONF_WCC	-1.9097	0.880	-2.171	0.031	-3.640	-0.179

Omnibus:	3.295	**Durbin-Watson:**	1.867
Prob(Omnibus):	0.193	**Jarque-Bera (JB):**	3.775
Skew:	0.029	**Prob(JB):**	0.151

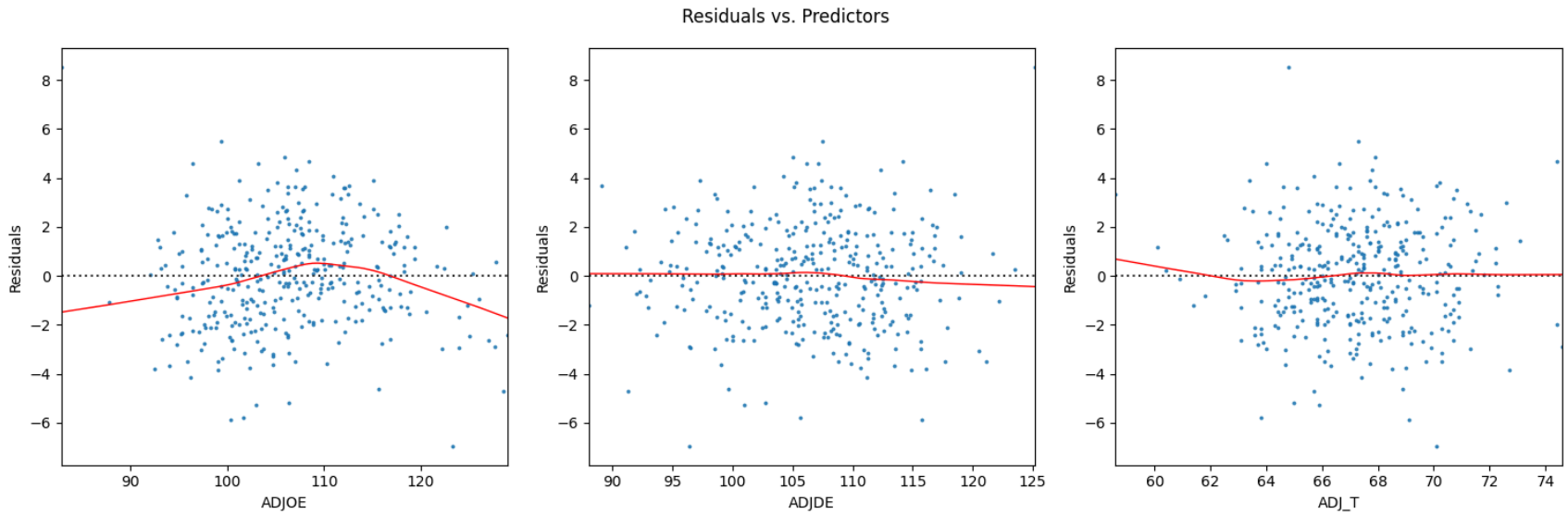
Kurtosis:	3.496	**Cond. No.**	7.09e+03

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 7.09e+03. This might indicate that there are
strong multicollinearity or other numerical problems.

(L) Linear

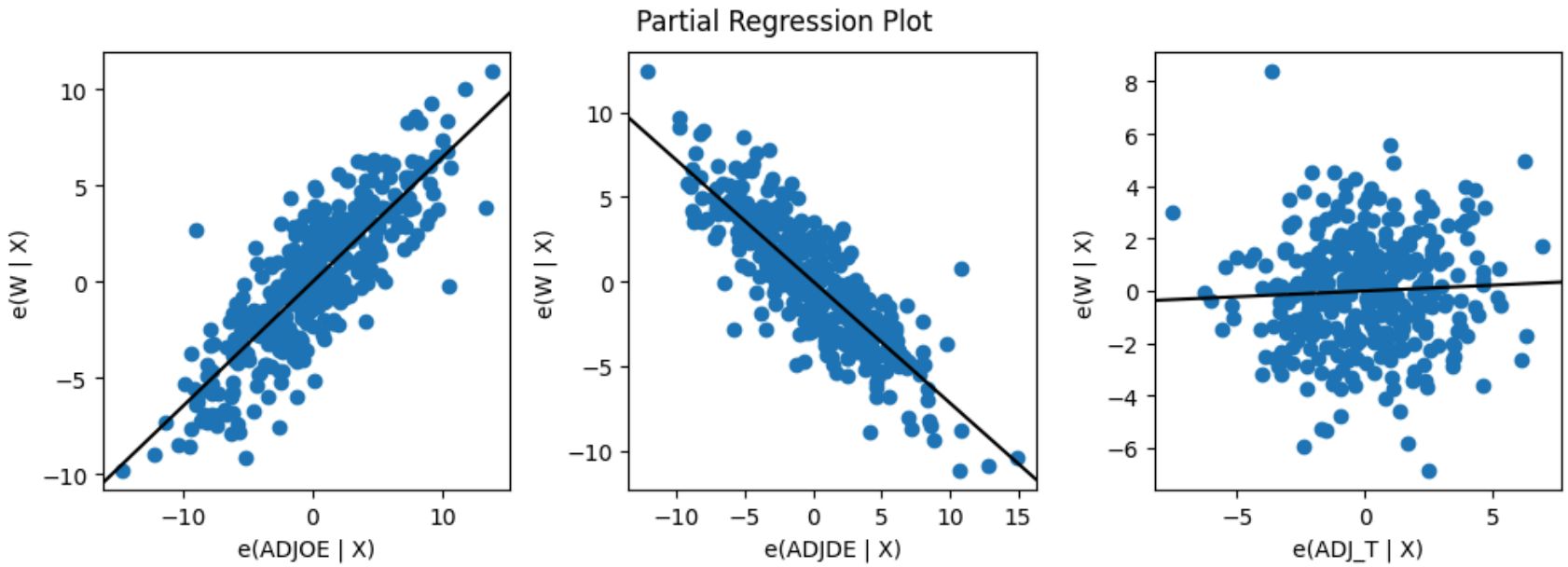
In [180…

```
# residual vs. predictor plots
def res_vs_pred(variable, ax):
    sns.residplot(x = basketball[variable],
                  y = basketball['residuals'],
                  lowess = True,
                  scatter_kws = {'s': 3},
                  line_kws = {'color': 'red', 'lw': 1},
                  ax = ax)
    ax.set_xlabel(variable)
    ax.set_ylabel("Residuals")

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
plt.suptitle("Residuals vs. Predictors")
res_vs_pred("ADJOE", axes[0])
res_vs_pred("ADJDE", axes[1])
res_vs_pred("ADJ_T", axes[2])
fig.tight_layout()
plt.show()
```



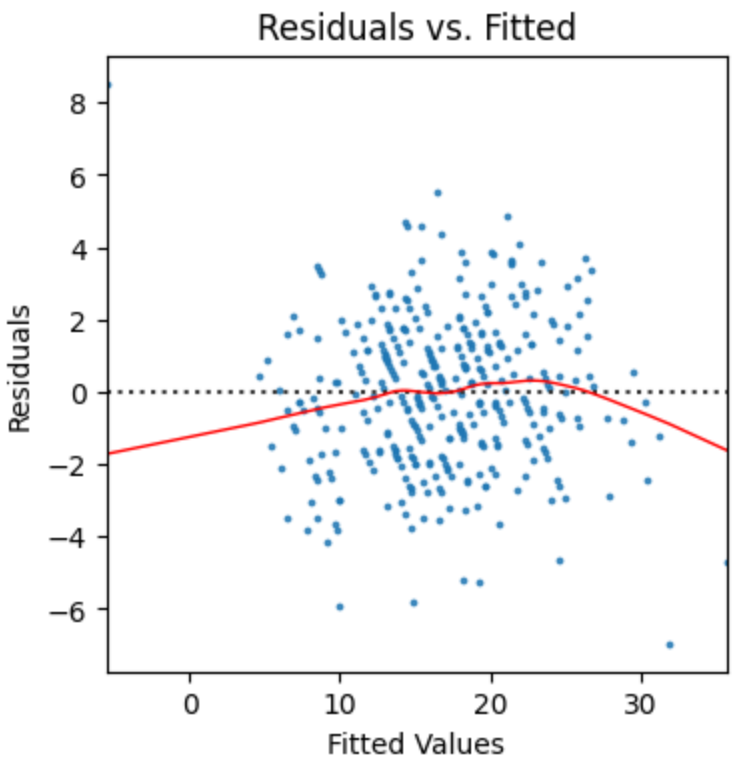
In [181…

```
fig = plt.figure(figsize = (10,10))
sm.graphics.plot_partregress_grid(res,
                                  exog_idx = list(basketball.columns[2:5]),
                                  grid = (3, 3),
                                  fig = fig)
fig.tight_layout()
plt.show()
```



In [182…

```
fig = plt.figure(figsize = (4, 4))
sns.residplot(x = basketball['fittedvalues'],
              y = basketball['residuals'],
              lowess = True,
              scatter_kws = {'s': 3},
              line_kws = {'color': 'red', 'lw': 1})
plt.title("Residuals vs. Fitted")
plt.ylabel("Residuals")
plt.xlabel("Fitted Values")
plt.show()
```



The assumptions seem to be met on ADJDE and ADJ_T as they are fitted in the model around 0 ADJOE seems be met as well there is not noticeable curvature. and residuals vs. fitted plots the red line curves away from 0 a good amount. The partial Regression Plots are good because they have a direction in trend ADJOE having a positive trend and ADJDE having a negative trend. ADJ_T seems to be close to not having a tread but it is positve.

(I) Independant

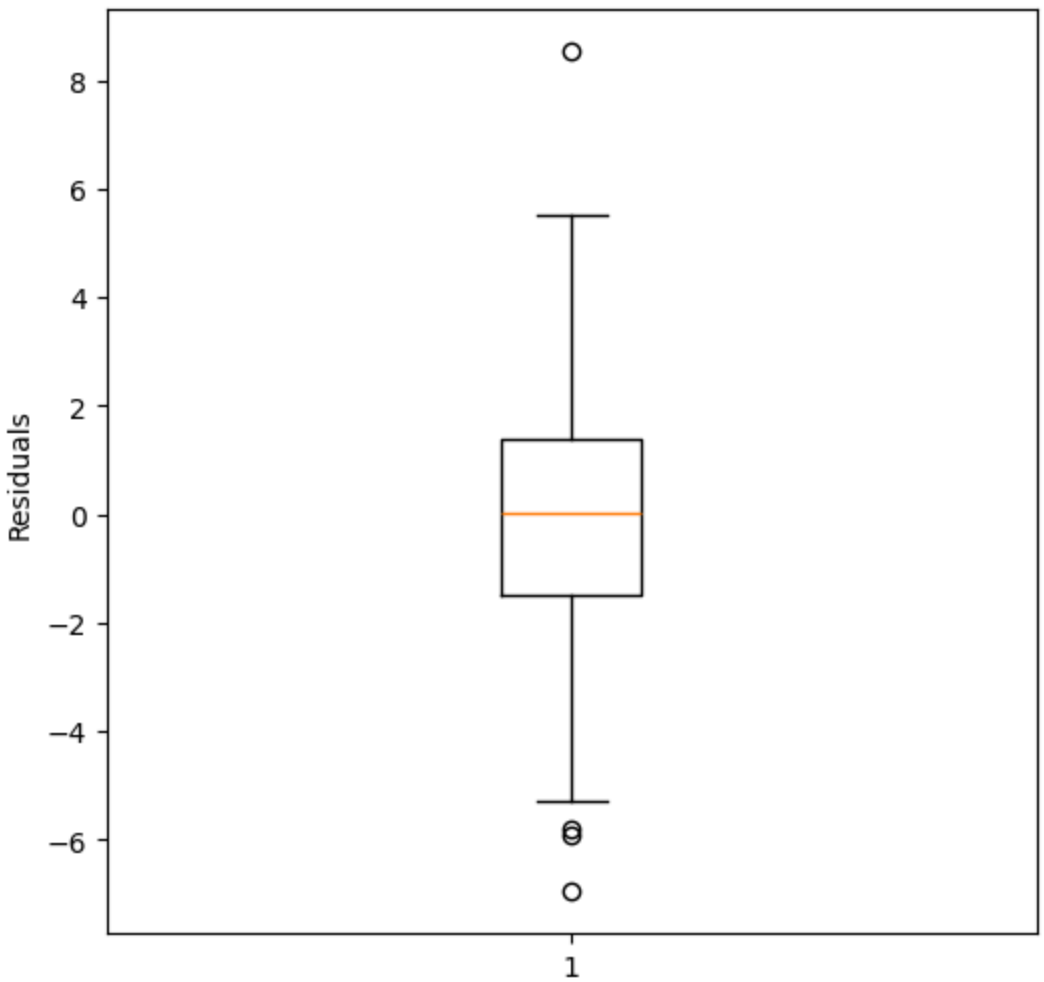
The assumption is met. Each observation is an individual team and none of them are repeated. We are also just looking at the overall data from just 2025 rather repeated yearly data.

(N) Normally distributed

In [183…

```
fig = plt.figure(figsize = (6, 6))
plt.boxplot(basketball['residuals'])
```

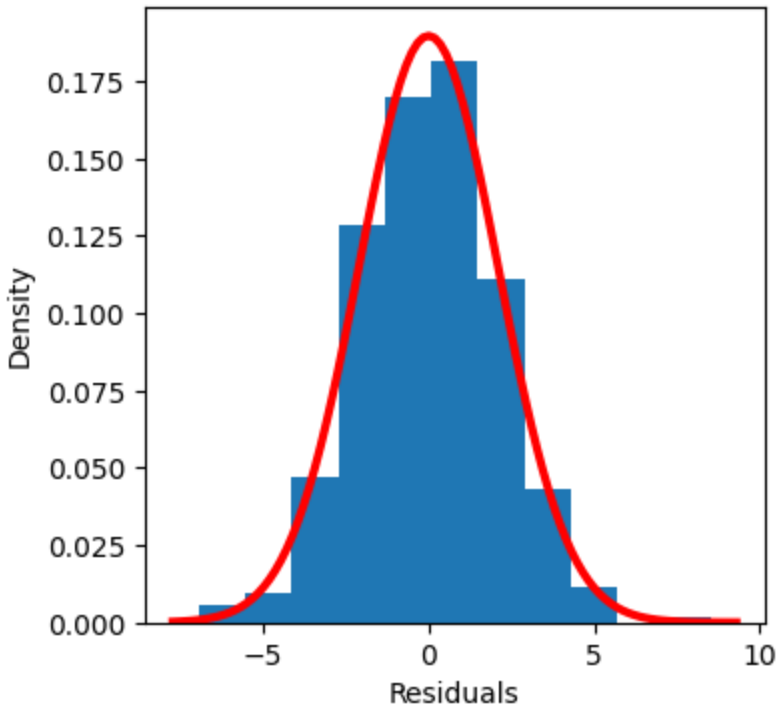
```
plt.ylabel("Residuals")
plt.show()
```



In [184…

```
# Diagnostic 2
fig = plt.figure(figsize = (4, 4))

# plot histogram (density = True so that it's on the same scale as the normal distribution)
plt.hist(basketball['residuals'],
         density = True,
         bins = 11)
plt.xlabel("Residuals")
```

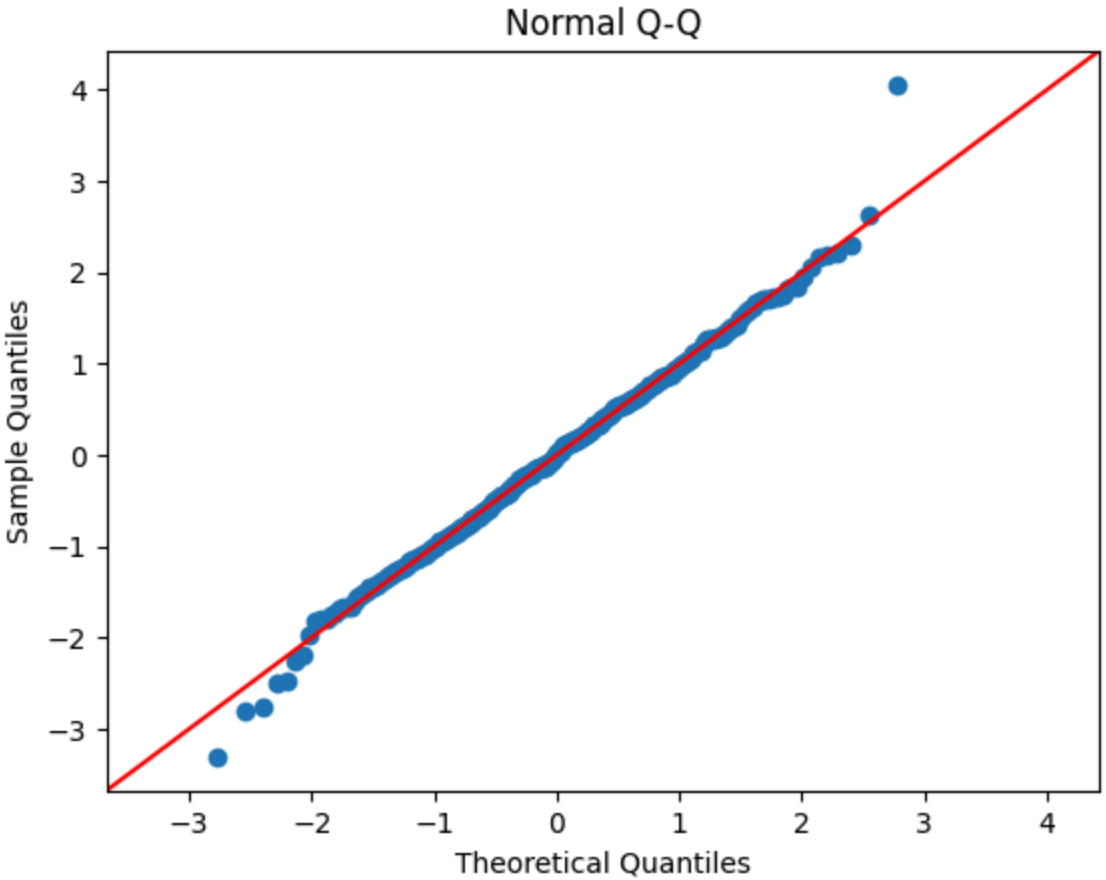
```
plt.ylabel("Density")

# calculate mean and standard deviation
mean = np.mean(basketball['residuals'])
sd = np.std(basketball['residuals'])

# generate x values to plot
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)

# plot normal distribution curve
plt.plot(x,
         stats.norm.pdf(x, mean, sd),
         color = 'r',
         lw = 3)
plt.show()
```



In [185…

```
# Diagnostic 3
sm.qqplot(basketball['residuals'],
```

```
            line = '45',
            fit = True)
plt.title("Normal Q-Q")
plt.show()
```



In [186…

```
# Diagnostic 4
stats.shapiro(basketball['residuals'])
```

Out[186…

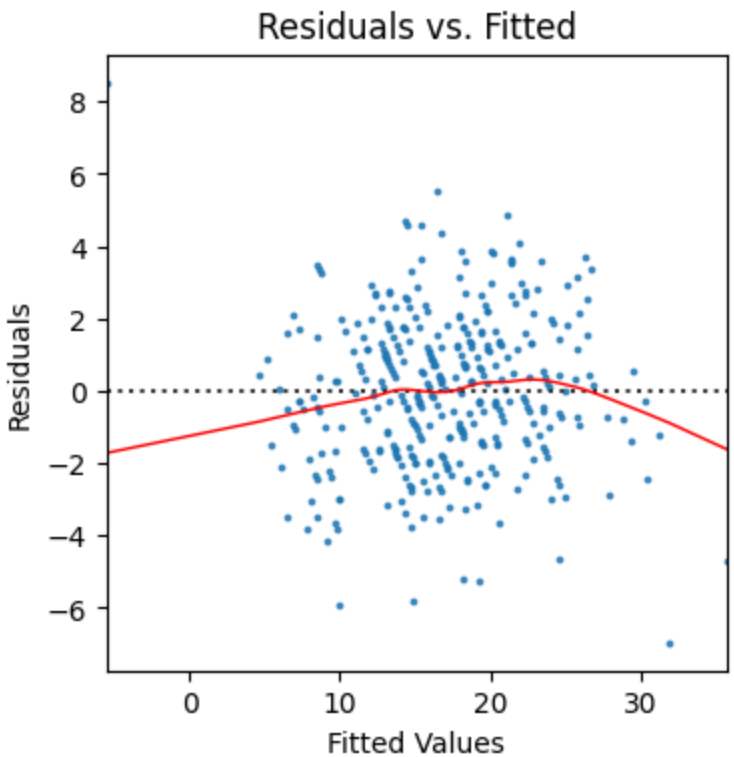
```
ShapiroResult(statistic=np.float64(0.9959337170232989), pvalue=np.float64(0.4730743989938066))
```

This assumption is met. There is possible tailing at the bottom of the QQ plot, some outliars in outside of the wicks in the box plot, the histogram seems to be slightly left skewed but the P-Value is higher than .05 so our assuption should be met.

(E) Equal/Constant Variance

In [187…

```
fig = plt.figure(figsize = (4, 4))
sns.residplot(x = basketball['fittedvalues'],
              y = basketball['residuals'],
              lowess = True,
              scatter_kws = {'s': 3},
              line_kws = {'color': 'red', 'lw': 1})
plt.title("Residuals vs. Fitted")
plt.ylabel("Residuals")
plt.xlabel("Fitted Values")
plt.show()
```



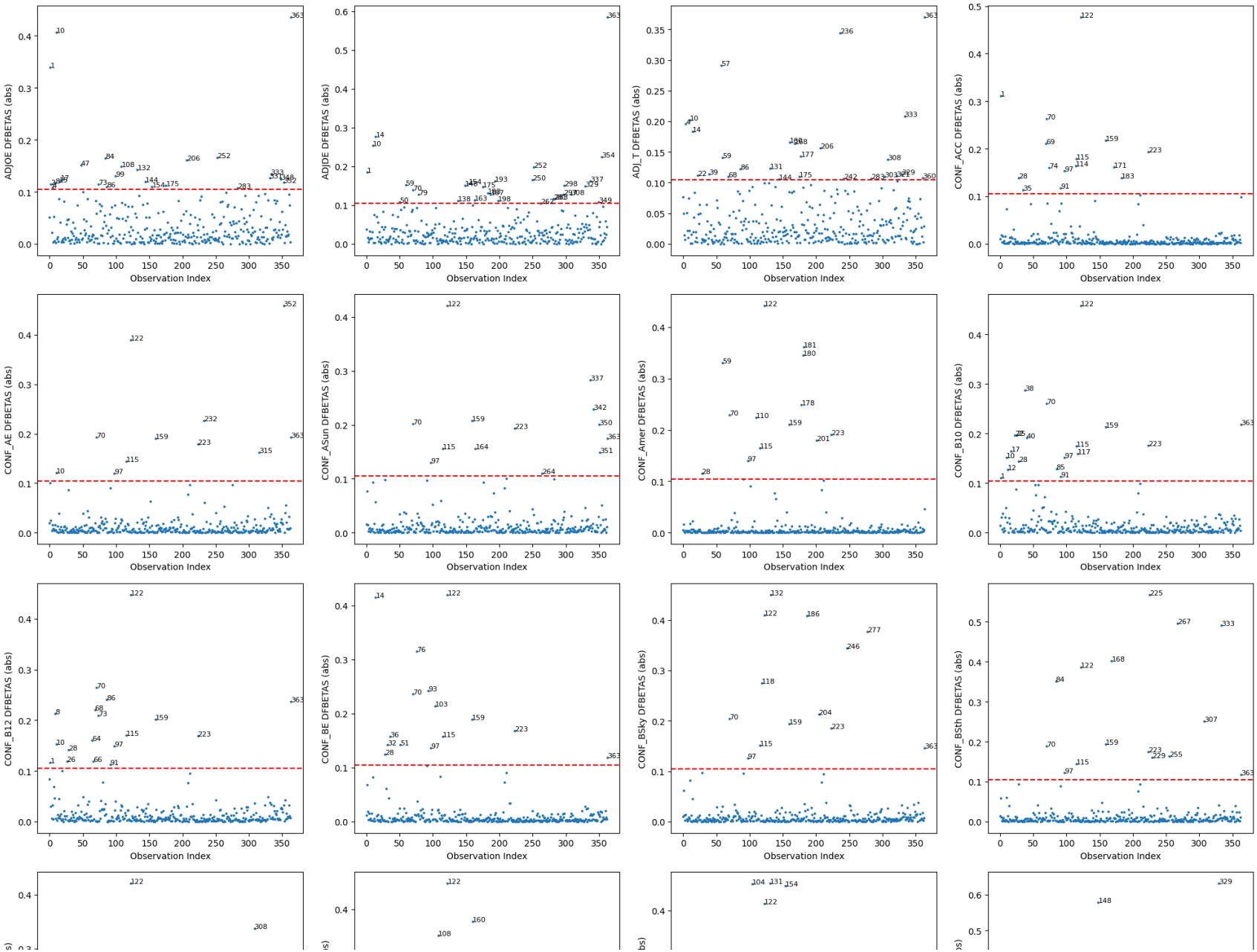
The assumption appears to be met. The vertical spread looks consistent throughout the plot.

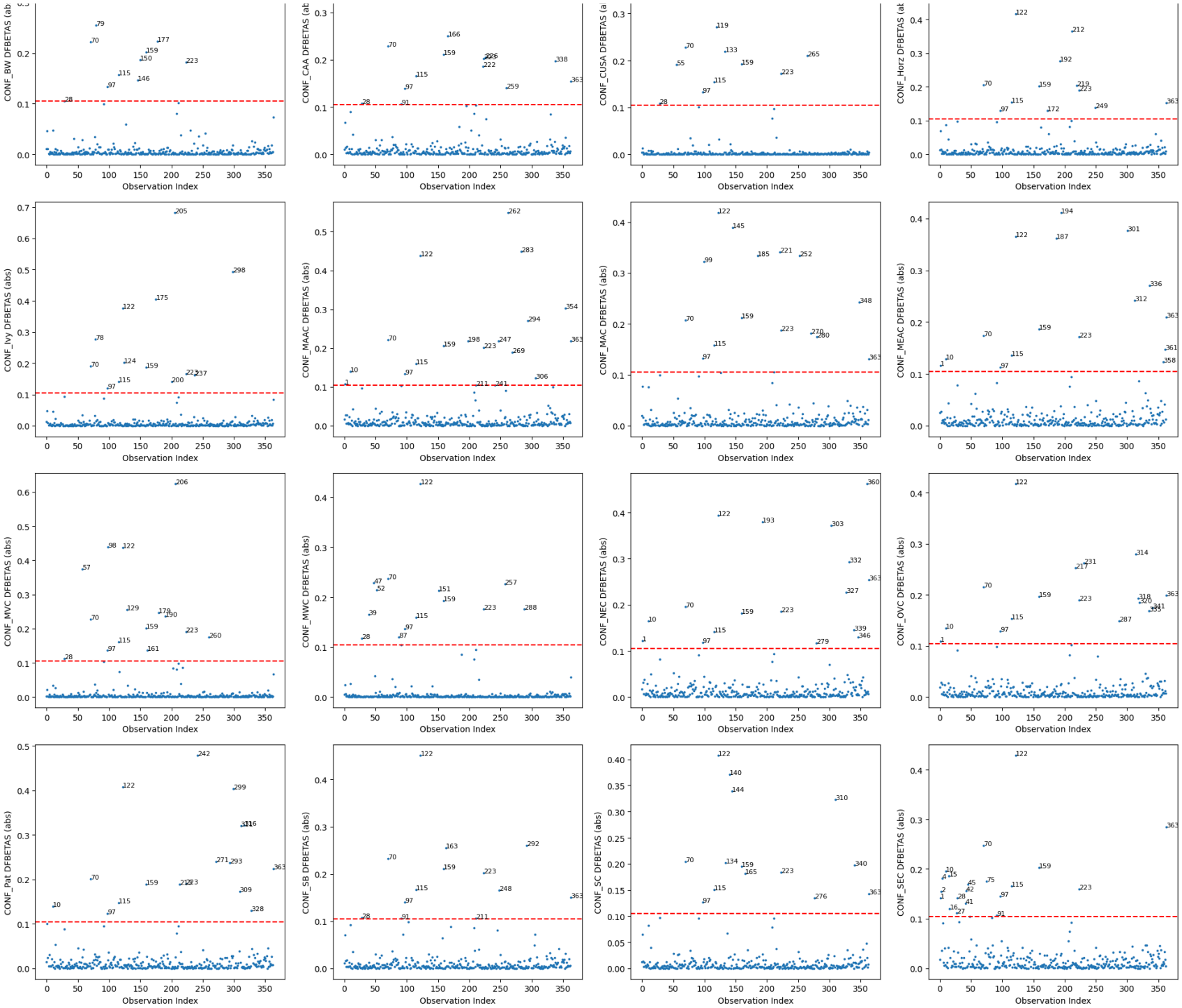
(A) Describes all Observations

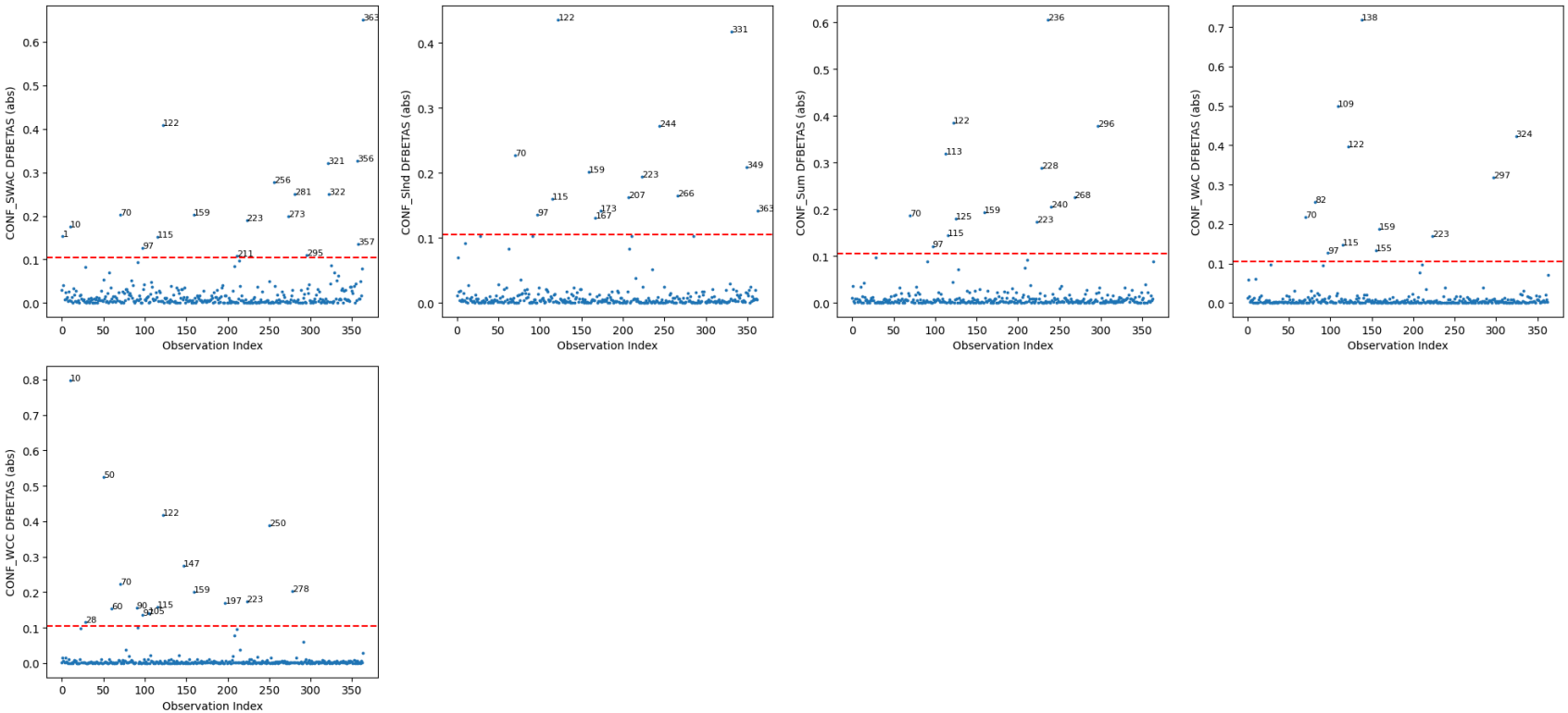
In [188…

```
def plot_dfbetas(column, ax, label_level=2 / np.sqrt(len(basketball))):
    if column not in res.model.exog_names:
        print(f"Error: '{column}' was not used in the regression model.")
        return
    idx = res.model.exog_names.index(column)
    dfbetas = res.get_influence().dfbetas[:, idx]
    obs_index = np.arange(len(dfbetas))  # ← fixes the size mismatch

    ax.set_ylabel(f"{column} DFBETAS (abs)")
    ax.set_xlabel("Observation Index")
    ax.scatter(obs_index, np.abs(dfbetas), s=3)
    ax.axhline(y=label_level, color='r', linestyle='dashed', label='Threshold')

    outlier_indices = np.where(np.abs(dfbetas) > label_level)[0]
    for i in outlier_indices:
        ax.annotate(str(i), (i, np.abs(dfbetas[i])), fontsize=8)
conf_cols = [col for col in res.model.exog_names if col.startswith('CONF_')]
all_cols = ['ADJOE', 'ADJDE', 'ADJ_T'] + conf_cols

n_cols = 4
n_rows = int(np.ceil(len(all_cols) / n_cols))

fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 5 * n_rows))
axes = axes.flatten()

for i, col in enumerate(all_cols):
    plot_dfbetas(col, axes[i])

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle("DFBETAS")
fig.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

DFBETAS


ADJOE DFBETAS (abs)
ADJDE DFBETAS (abs)
ADJ_T DFBETAS (abs)
CONF_ACC DFBETAS (abs)
CONF_AE DFBETAS (abs)
CONF_ASun DFBETAS (abs)
CONF_Amer DFBETAS (abs)
CONF_B10 DFBETAS (abs)
CONF_B12 DFBETAS (abs)
CONF_BE DFBETAS (abs)
CONF_BSky DFBETAS (abs)
CONF_BSth DFBETAS (abs)
Observation Index

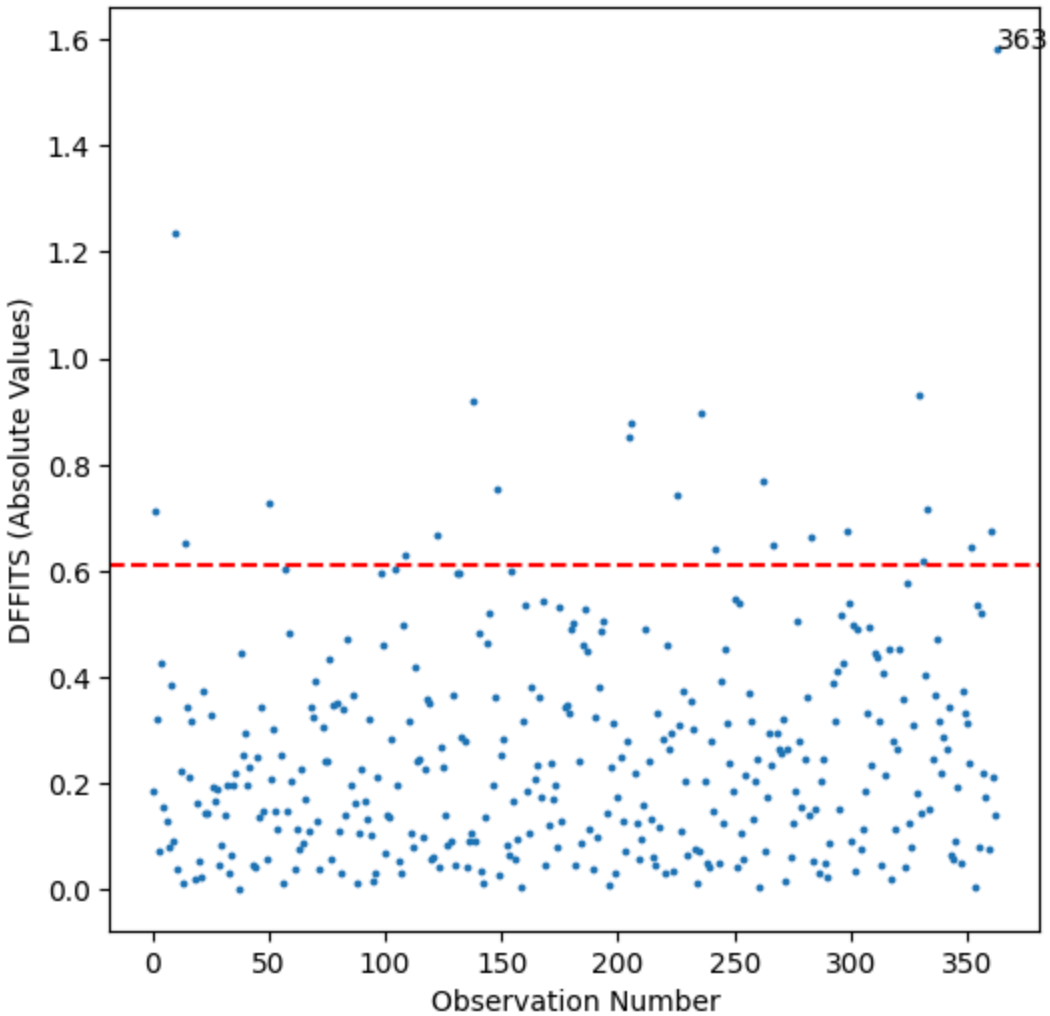

CONF_BW DFBETAS (abs)
CONF_CAA DFBETAS (abs)
CONF_CUSA DFBETAS (abs)
CONF_Horz DFBETAS (abs)
CONF_Ivy DFBETAS (abs)
CONF_MAAC DFBETAS (abs)
CONF_MAC DFBETAS (abs)
CONF_MEAC DFBETAS (abs)
CONF_MVC DFBETAS (abs)
CONF_MWC DFBETAS (abs)
CONF_NEC DFBETAS (abs)
CONF_OVC DFBETAS (abs)
CONF_Pat DFBETAS (abs)
CONF_SB DFBETAS (abs)
CONF_SC DFBETAS (abs)
CONF_SEC DFBETAS (abs)
Observation Index



In [189…

```
# DFFITS
basketball['dffits'] = res.get_influence().dffits[0]

fig, ax = plt.subplots(figsize = (6, 6))
ax.set_ylabel("DFFITS (Absolute Values)")
ax.set_xlabel("Observation Number")

# scatter + cutoff line
ax.scatter(basketball.index,
           np.abs(basketball['dffits']),
           s = 3)
ax.axhline(y = 2 * np.sqrt(len(res.params) / len(basketball)),
           color = 'r',
           linestyle = 'dashed')

# Change the y_cutoff value to whatever number you want - all the
# points above that value will be labeled with their row number
y_cutoff_dffits = 1.3
```

```
outliers_dffits = basketball[np.abs(basketball['dffits']) > y_cutoff_dffits]
for i in outliers_dffits.index:
    ax.annotate(str(i), (i, np.abs(basketball['dffits'][i])))

plt.show()
```



The assumption is met. In each plot, there are points that fall above the red line, all look good except for the plot 363 is questionable but after removing it from the model and checking all assumption not much changes so we assume that it is not an outliar.

In [190…
```
basketball.iloc[363]
```

Out[190…
```
W                     3
CONF                SWAC
ADJOE               82.9
ADJDE              125.2
ADJ_T               64.8
residuals       8.525737
fittedvalues   -5.525737
dffits          1.582684
Name: 363, dtype: object
```

We removed dot 363 to see if it had a significant change on the chart. We found that the assumptions aren't changed much so we decided to keep dot 363 in the model as normal.

In [191…
```
# 1. Filter the dataframe for rows where 'fittedvalues' is less than 0
negative_preds = basketball[basketball['fittedvalues'] < 0]

# 2. Count how many there are
print(f"Number of teams with negative predicted wins: {len(negative_preds)}")

# 3. View the data for these teams
print(negative_preds[['fittedvalues', 'residuals', 'W']])
```

```
Number of teams with negative predicted wins: 1
     fittedvalues  residuals  W
363     -5.525737   8.525737  3
```

This is seemed outlier. It looks like this team won 3 games but they were actually predicted to win -5.5 games which is physically impossible. This is concerning in our dataset. the model is good at predicting all teams but this one. I have taken a look and this is the only team with negative wins predicted. overall though our model is good at predicting wins. But after removing it from the model and checking assumptions this is in fact not an outliar.

(R) Additional predictor variables are not required

This assumption is reasonably met. We are taking into account several different predictor variables that we have found have a significant statistical correlation with the wins. Years of experiece the players have, different injuries that could have occured in the season. These would be nice to have in the model to see what significance they have.

No Multicolinearity

In [192…

```
basketball_vifs = pd.DataFrame()
basketball_vifs['Feature'] = X.columns[1:4]
basketball_vifs['VIF'] = [vif(X, i) for i in range(1, 4)]

print("Max = ", max(basketball_vifs['VIF']))
print("Mean = ", np.mean(basketball_vifs['VIF']))
basketball_vifs
```

```
Max =  2.4585172338469685
Mean =  1.9548258926465607
```

Out[192…

	Feature	VIF
0	ADJOE	2.458517
1	ADJDE	2.306132
2	ADJ_T	1.099829

This assumption is not violated. The VIF for all are around 2 which is pretty good. There doesn't really seem to be multicollinearity. They all seem to give something very different in the data. and that each varible gives us something different and independant from the other varibles.

Overall, based on our diagnostics our assumptions were satisfied. Normal distribution seems to be met. And after Removing and checking the DFBETAS and DFFITS again we see that 363 doesn't effect the model significantly, so we have decided to keep it in.

Variable Selection

Create and structure Data

In [193…

```
basket.head()
```

Out[193…

	W	ADJOE	ADJDE	ADJ_T	CONF_ACC	CONF_AE	CONF_ASun	CONF_Amer	CONF_B10	CONF_B12	…	CONF_OVC
0	30	124.8	88.0	61.4	0	0	0	0	0	1	…	0
1	31	128.5	91.3	65.7	1	0	0	0	0	0	…	0
2	28	129.0	93.7	67.8	0	0	0	0	0	0	…	0
3	30	127.7	94.0	69.5	0	0	0	0	0	0	…	0
4	25	127.6	96.4	74.6	0	0	0	0	0	0	…	0

5 rows × 34 columns

In [194…

```
# Split out X and y
y = basket['W']
X = basket[['ADJOE',
  'ADJDE',
  'ADJ_T',
  'CONF_ACC',
  'CONF_AE',
  'CONF_ASun',
  'CONF_Amer',
  'CONF_B10',
  'CONF_B12',
  'CONF_BE',
  'CONF_BSky',
  'CONF_BSth',
  'CONF_BW',
  'CONF_CAA',
  'CONF_CUSA',
  'CONF_Horz',
  'CONF_Ivy',
  'CONF_MAAC',
  'CONF_MAC',
  'CONF_MEAC',
  'CONF_MVC',
  'CONF_MWC',
  'CONF_NEC',
  'CONF_OVC',
  'CONF_Pat',
  'CONF_SB',
```

```
  'CONF_SC',
  'CONF_SEC',
  'CONF_SWAC',
  'CONF_Slnd',
  'CONF_Sum',
  'CONF_WAC',
  'CONF_WCC']]
X.head()
```

Out[194…

	ADJOE	ADJDE	ADJ_T	CONF_ACC	CONF_AE	CONF_ASun	CONF_Amer	CONF_B10	CONF_B12	CONF_BE	…	CONI
0	124.8	88.0	61.4	0	0	0	0	0	1	0	…	
1	128.5	91.3	65.7	1	0	0	0	0	0	0	…	
2	129.0	93.7	67.8	0	0	0	0	0	0	0	…	
3	127.7	94.0	69.5	0	0	0	0	0	0	0	…	
4	127.6	96.4	74.6	0	0	0	0	0	0	0	…	

5 rows × 33 columns

In [195…

```
# Tell python which dummy variables belong to the same categorical variable:
feature_groups = [[0], [1], [2], list(range(3, 33))]
```

We just created dummy varibles so we can fit our subsets models and lasso models.

Best Subsets

In [196…

```
# create function for plotting
def plot_model_selection_1se(selection_result, figsize = (8, 5)):
    """
    Plot model selection performance vs number of features
    using the 1-SE rule.

    Parameters
    ----------
    selection_result : object
        Must have method get_metric_dict() returning a dictionary
```

```
        with keys including:
            - "feature_names"
            - "avg_score"
            - "std_err"

    figsize : tuple
        Figure size for plot.
    """

    # Dictionary with key-value pairs
    # Key is an ID number for the subset
    # Value holds things like the number of variables in the subset, PMSE values, SD, etc.
    metric_dict = selection_result.get_metric_dict()

    # Convert metric_dict to a data frame (one row per subset)
    # orient = "index" tells python to use the keys as row names in the data frame
    results_df = pd.DataFrame.from_dict(metric_dict, orient="index")

    # number of features in each subset
    results_df["k"] = results_df["feature_names"].apply(len)

    # bounds for uncertainty
    results_df["upper"] = results_df["avg_score"] + results_df["std_err"]
    results_df["lower"] = results_df["avg_score"] - results_df["std_err"]

    # best subset within each k (max avg_score because it's negative MSE)
    best_by_k = (
        results_df.sort_values("avg_score", ascending=False)
        .groupby("k", as_index=False)
        .head(1)
        .sort_values("k")
    )

    # 1-SE rule: choose the smallest k whose performance is within 1 SE of the best
    best_row = best_by_k.loc[best_by_k["avg_score"].idxmax()]
    threshold = best_row["avg_score"] - best_row["std_err"]   # within 1 SE of best (higher is better)

    k_1se = best_by_k.loc[best_by_k["avg_score"] >= threshold, "k"].min()
    row_1se = best_by_k[best_by_k["k"] == k_1se].iloc[0]

    plt.figure(figsize=figsize)
    plt.fill_between(best_by_k["k"], best_by_k["lower"], best_by_k["upper"], alpha=0.2, lw=1)
```

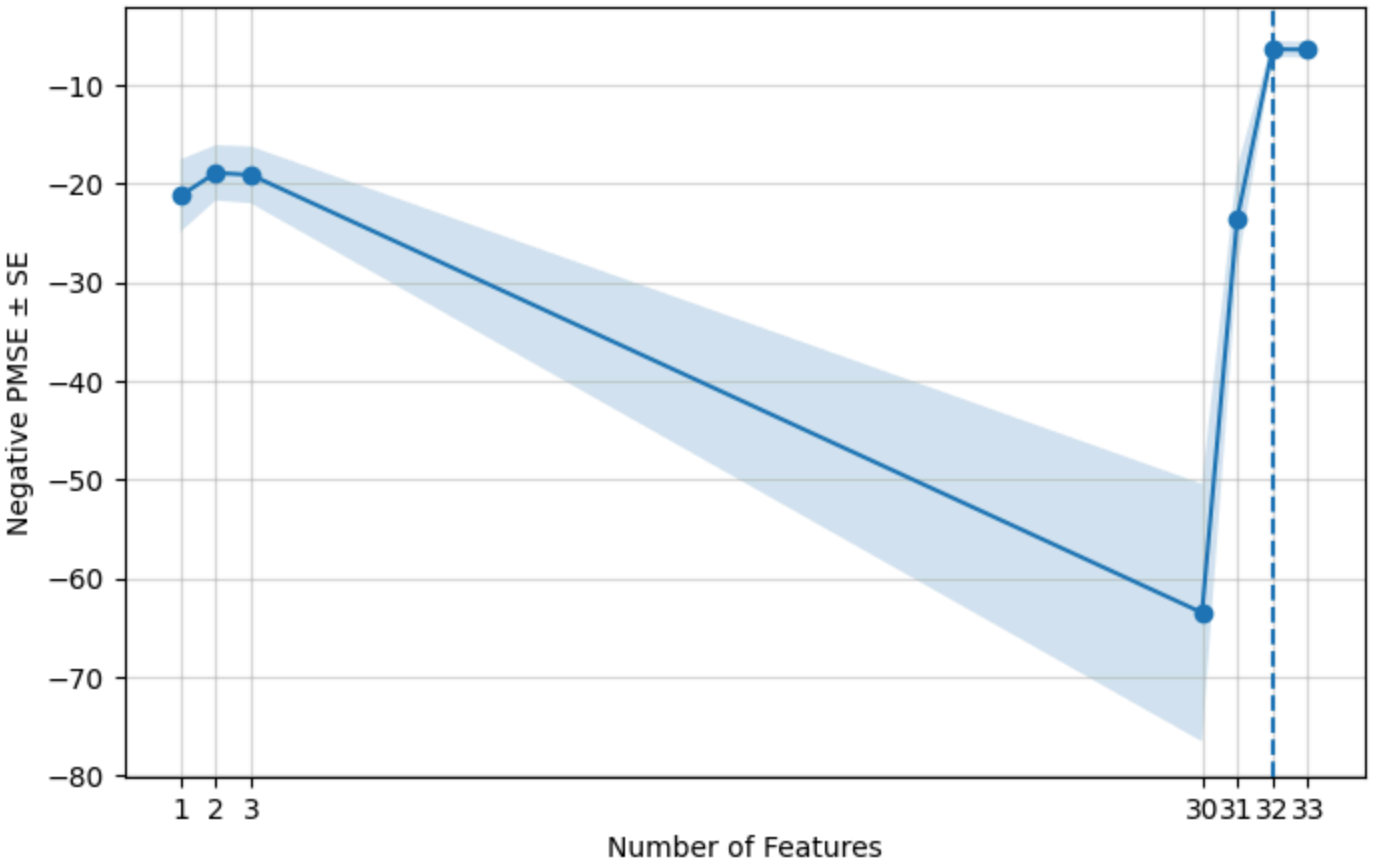
```
    plt.plot(best_by_k["k"], best_by_k["avg_score"], marker="o")
    plt.axvline(row_1se["k"], linestyle="--")
    plt.xlabel("Number of Features")
    plt.ylabel("Negative PMSE ± SE")
    plt.grid(alpha=0.5)
    plt.xticks(ticks=best_by_k["k"], labels=best_by_k["k"])
    plt.show()
```

In [197…

```
best_subsets_selection = EFS(LinearRegression(fit_intercept = True),
                             min_features = 1,
                             max_features = len(feature_groups),  # defined above
                             scoring = 'neg_mean_squared_error',
                             print_progress = False,
                             feature_groups = feature_groups,  # defined above
                             cv = 5)

best_subsets = best_subsets_selection.fit(X, y)
```

In [198…

```
plot_model_selection_1se(best_subsets)
```



This is showing all of the models that we can use and the best fits. As you can tell we can't fit a model with 4 - 29 varibles. It appears like a weird chart but this dataset requires all conferences or no conferences. 32 is the 1-SE rule.

In [199…

```
best_subsets_dict = best_subsets_selection.get_metric_dict()
best_subsets_df = pd.DataFrame.from_dict(best_subsets_dict, orient="index")

# number of features in each model
best_subsets_df["n_features"] = best_subsets_df["feature_names"].apply(len)

# --- choose which models you want to see based on the number of variables included ---
best_subsets_min_k = 32
best_subsets_max_k = 32
# ------------------------------------------------------------------------------------
```

```
# restrict to desired k range
best_subsets_df_k = best_subsets_df[(best_subsets_df["n_features"] >= best_subsets_min_k) &
                    (best_subsets_df["n_features"] <= best_subsets_max_k)].copy()

# for each k, keep only the best-performing model
best_subsets_df_per_k = (best_subsets_df_k.sort_values("avg_score", ascending=False)
                                                        .groupby("n_features", as_index = False)
                                                        .head(1)
                                                        .sort_values("n_features"))

# display nicely
pd.set_option("display.max_colwidth", None)
print(best_subsets_df_per_k[["n_features", "feature_names", "avg_score"]])
pd.reset_option("display.max_colwidth")
```

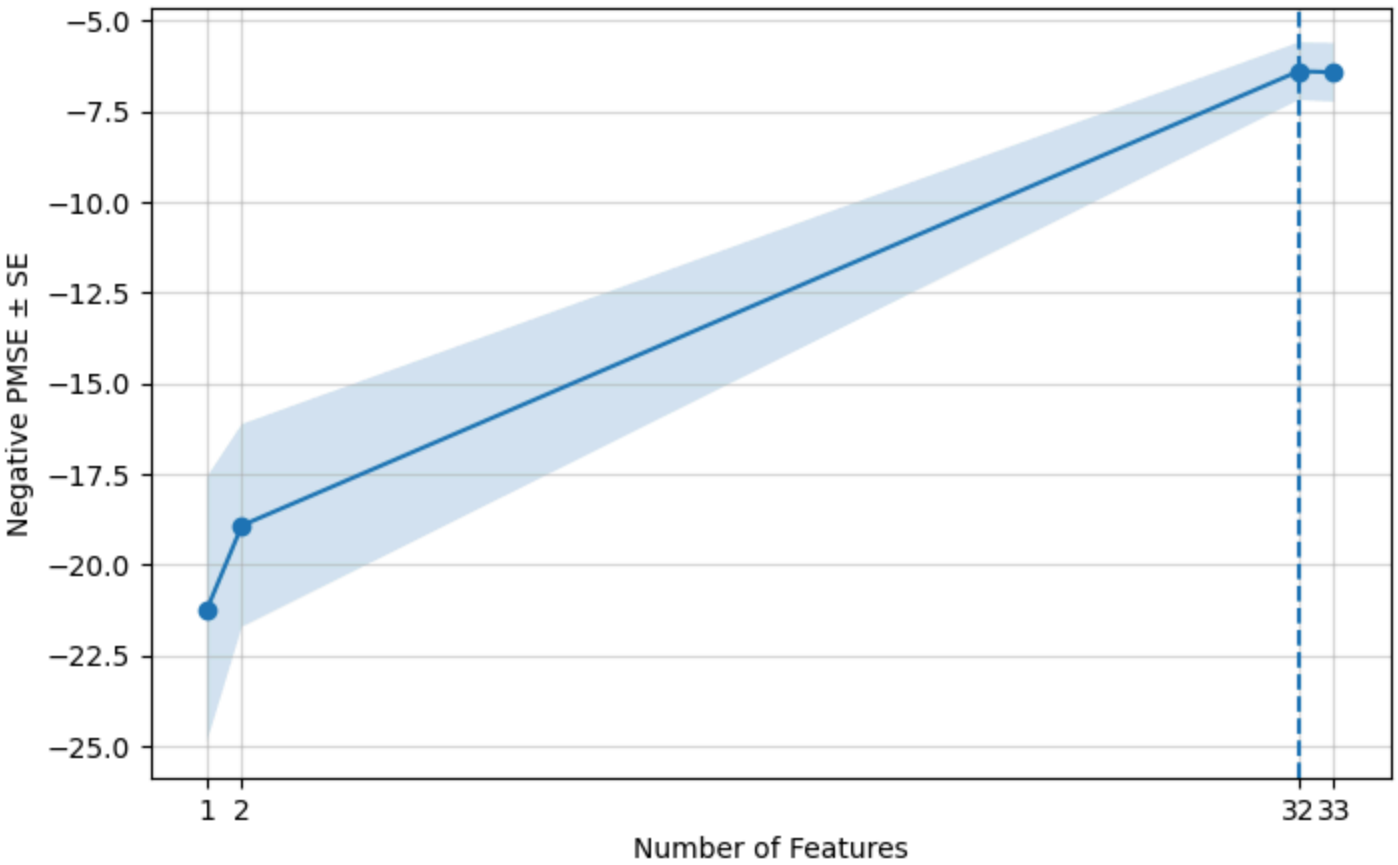
```
    n_features  \
11          32


feature_names  \
11  (ADJOE, ADJDE, CONF_ACC, CONF_AE, CONF_ASun, CONF_Amer, CONF_B10, CONF_B12, CONF_BE, CONF_BSky, CONF_BS
th, CONF_BW, CONF_CAA, CONF_CUSA, CONF_Horz, CONF_Ivy, CONF_MAAC, CONF_MAC, CONF_MEAC, CONF_MVC, CONF_MWC,
CONF_NEC, CONF_OVC, CONF_Pat, CONF_SB, CONF_SC, CONF_SEC, CONF_SWAC, CONF_Slnd, CONF_Sum, CONF_WAC, CONF_WC
C)

    avg_score
11  -6.395977
```

It looks like the best model for us would have 32 varibles in the model removing ADJ_T. That is our suggested 1-SE number and we are going to use that amount of varibles.

In [200…

```
seqrep_selection = SFS(LinearRegression(fit_intercept = True),
                       k_features = (1, len(feature_groups)),  # defined above
                       forward = True,
                       floating = True,
                       scoring = 'neg_mean_squared_error',
                       feature_groups = feature_groups,  # defined above
                       cv = 5)

seqrep = seqrep_selection.fit(X, y)
```

In [201…
```
plot_model_selection_1se(seqrep)
```



This is showing all of the models that we can use and the best fits. As you can tell we can't fit a model with 3 - 31 varibles. It appears like a weird chart but this dataset requires all conferences or no conferences. 32 is the 1-SE rule.

In [202…
```
seqrep_selection_dict = seqrep_selection.get_metric_dict()
seqrep_selection_df = pd.DataFrame.from_dict(seqrep_selection_dict, orient="index")

# number of features in each model
seqrep_selection_df["n_features"] = seqrep_selection_df["feature_names"].apply(len)

# --- choose which models you want to see based on the number of variables included ---
seqrep_selection_min_k = 32
```

```
seqrep_selection_max_k = 32
# ------------------------------------------------------------------------------------

# restrict to desired k range
seqrep_selection_df_k = seqrep_selection_df[(seqrep_selection_df["n_features"] >= seqrep_selection_min_k) &
                    (seqrep_selection_df["n_features"] <= seqrep_selection_max_k)].copy()

# for each k, keep only the best-performing model
seqrep_selection_df_per_k = (seqrep_selection_df_k.sort_values("avg_score", ascending=False)
                                                   .groupby("n_features", as_index = False)
                                                   .head(1)
                                                   .sort_values("n_features"))

# display nicely
pd.set_option("display.max_colwidth", None)
print(seqrep_selection_df_per_k[["n_features", "feature_names", "avg_score"]])
pd.reset_option("display.max_colwidth")
```

```
   n_features  \
3          32


feature_names  \
3  (ADJOE, ADJDE, CONF_ACC, CONF_AE, CONF_ASun, CONF_Amer, CONF_B10, CONF_B12, CONF_BE, CONF_BSky, CONF_BSt
h, CONF_BW, CONF_CAA, CONF_CUSA, CONF_Horz, CONF_Ivy, CONF_MAAC, CONF_MAC, CONF_MEAC, CONF_MVC, CONF_MWC, C
ONF_NEC, CONF_OVC, CONF_Pat, CONF_SB, CONF_SC, CONF_SEC, CONF_SWAC, CONF_Slnd, CONF_Sum, CONF_WAC, CONF_WC
C)

   avg_score
3  -6.395977
```

It looks like the best model for us would have 32 varibles in the model removing ADJ_T. That is our suggested 1-SE number and we are going to use that amount of varibles.

Group LASSO

In [203…

```
pipe_gl = make_pipeline(
    StandardScaler(),  # standardize predictors
    GroupLasso(
        groups = feature_groups,
        alpha = 1.0,  # equivalent to lambda, a placeholder that will later be tuned by CV
```

```
        fit_intercept = True  # add an intercept since feature_groups doesn't have one (and shouldn't)
    )
)
```

In [204…

```
# Create our own cross-validation since GroupLassoCV doesn't exist
alpha_grid = np.logspace(-4, 4, 100)  # values of alpha/lambda to try, YOU alter this as needed based on t

gscv = GridSearchCV(
    estimator = pipe_gl,  # use the pipe above with GroupLasso
    param_grid = {"grouplasso__alpha": alpha_grid},  # "grouplasso" name comes from make_pipeline
    scoring = "neg_mean_squared_error",  # negative PMSE
    cv = 5,  # 5-fold cross-validation
    n_jobs = -1,  # run this in parallel using *all* cores
    refit = False  # we'll refit this manually after the 1-SE selection
)

gscv.fit(X, y)
```

Out[204…

```
▸ GridSearchCV
▸ estimator: Pipeline
▸ StandardScaler
▸ GroupLasso
```

In [205…

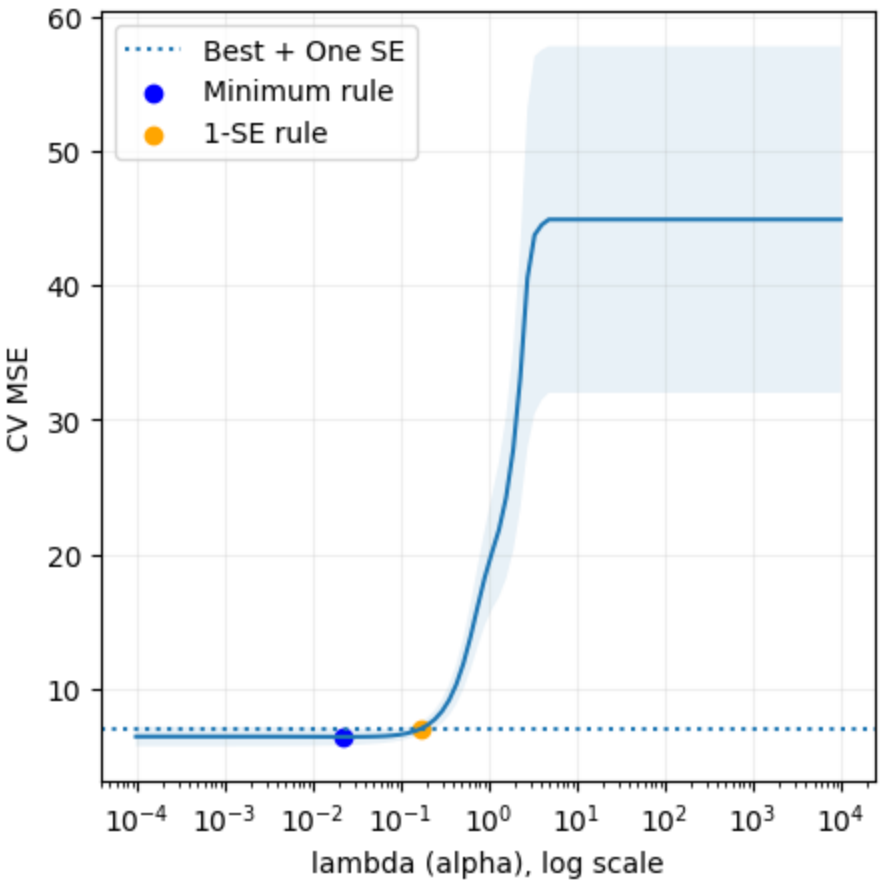
```
cv_res = pd.DataFrame(gscv.cv_results_)
cv_res["mean_mse"] = -cv_res["mean_test_score"]  # positive MSE
cv_res["se_mse"] = cv_res["std_test_score"] / np.sqrt(gscv.cv)  # standard error across folds
cv_res["alpha"] = cv_res["param_grouplasso__alpha"].astype(float)

# 1-SE rule: largest alpha within 1 SE of the minimum MSE
best_idx = cv_res["mean_mse"].idxmin()
best_mse = cv_res.loc[best_idx, "mean_mse"]
best_se  = cv_res.loc[best_idx, "se_mse"]
threshold = best_mse + best_se

alpha_1se = cv_res.loc[cv_res["mean_mse"] <= threshold, "alpha"].max()
alpha_min = cv_res.loc[best_idx, "alpha"]
```

```
print("Minimum-rule alpha:", alpha_min)
print("1-SE-rule alpha:", alpha_1se)
print("Best mean CV MSE:", best_mse)
```

```
Minimum-rule alpha: 0.022051307399030457
1-SE-rule alpha: 0.1707352647470692
Best mean CV MSE: 6.408545959674579
```

In [206…

```
plot_df = cv_res.sort_values("alpha")

plt.figure(figsize = (5, 5))
plt.plot(plot_df["alpha"],
         plot_df["mean_mse"])
plt.fill_between(
    plot_df["alpha"],
    plot_df["mean_mse"] - plot_df["se_mse"],
    plot_df["mean_mse"] + plot_df["se_mse"],
    alpha = 0.1
)
plt.axhline(threshold,
            linestyle = "dotted",
            label = "Best + One SE")
plt.scatter([alpha_min],
            [best_mse],
            color = "blue",
            label = "Minimum rule")
plt.scatter([alpha_1se],
            [plot_df.loc[plot_df["alpha"] == alpha_1se, "mean_mse"].values[0]],
            color = "orange",
            label = "1-SE rule")
plt.xscale("log")
plt.xlabel("lambda (alpha), log scale")
plt.ylabel("CV MSE")
plt.legend()
plt.grid(alpha = 0.2)
plt.show()
```



In [207…

```
# Fit final model at alpha_1se
final = make_pipeline(
    StandardScaler(),
    GroupLasso(
        groups = feature_groups,
        alpha = alpha_1se,
        fit_intercept = True,
    )
)
final.fit(X, y)

coefs = final.named_steps["grouplasso"].coef_.ravel()
coef_table = (
    pd.DataFrame({"feature": X.columns, "coef": coefs})
```

```
        .assign(abs_coef = lambda d: d["coef"].abs())
        .sort_values("abs_coef", ascending = False)
        .drop(columns = "abs_coef")
)
print(coef_table)
```

```
      feature      coef
0       ADJOE  4.410383
1       ADJDE -4.215526
8    CONF_B12 -1.703787
27   CONF_SEC -1.599234
7    CONF_B10 -1.503032
3    CONF_ACC -1.039118
28  CONF_SWAC  0.973738
9     CONF_BE -0.941794
22   CONF_NEC  0.921578
23   CONF_OVC  0.901452
19  CONF_MEAC  0.839946
4     CONF_AE  0.837116
17  CONF_MAAC  0.821517
24   CONF_Pat  0.798733
26    CONF_SC  0.640706
18   CONF_MAC  0.619940
13   CONF_CAA  0.595774
15  CONF_Horz  0.581938
11  CONF_BSth  0.541842
21   CONF_MWC -0.536108
5   CONF_ASun  0.532945
25    CONF_SB  0.525973
10  CONF_BSky  0.439247
29  CONF_Slnd  0.404752
12    CONF_BW  0.400701
32   CONF_WCC -0.374314
30   CONF_Sum  0.255451
20   CONF_MVC  0.173564
31   CONF_WAC  0.154775
6   CONF_Amer  0.071150
16   CONF_Ivy  0.053627
14  CONF_CUSA  0.053193
2       ADJ_T  0.000000
```

Once again we are going to be looking at all those 32 varibles to predict wins. As ADJ_T will be left out showing 0s in this model.

In every single method it was reccommending us to take out ADJ_T from the dataset. While leaving all other varibles in the dataset. The methods all agreed which is really simple for us to predict now. We plan to move on with all varibles minus ADJ_T as it's not needed in any method. We felt there was no reason to override our any methods as they were all constant in what they were trying to tell us.

In [208…

```
basket = pd.get_dummies(basketball, columns=['CONF'], dtype=int)

# 2. Define your numeric efficiency features
features_base = ['ADJOE', 'ADJDE']

# 3. Automatically grab all the newly created conference columns
# This saves you from typing out every 'CONF_...' name manually
features_conf = [col for col in basket.columns if col.startswith('CONF_')]

# 4. Combine them for your final X
X = basket[features_base + features_conf]
y = basket['W'] # Double check if your column is 'W' or 'wins'

# 5. Add constant and fit
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()

print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      W   R-squared:                       0.871
Model:                            OLS   Adj. R-squared:                  0.859
Method:                 Least Squares   F-statistic:                     69.93
Date:                Wed, 25 Mar 2026   Prob (F-statistic):          2.00e-127
Time:                        12:22:51   Log-Likelihood:                -787.42
No. Observations:                 364   AIC:                             1641.
Df Residuals:                     331   BIC:                             1769.
Df Model:                          32
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         23.8405      4.054      5.881      0.000      15.866      31.815
ADJOE          0.6485      0.024     27.469      0.000       0.602       0.695
ADJDE         -0.7139      0.026    -27.167      0.000      -0.766      -0.662
CONF_A10      -1.2475      0.577     -2.161      0.031      -2.383      -0.112
CONF_ACC      -6.2172      0.566    -10.980      0.000      -7.331      -5.103
CONF_AE        5.9506      0.747      7.963      0.000       4.481       7.421
CONF_ASun      3.2058      0.668      4.802      0.000       1.892       4.519
CONF_Amer     -0.1423      0.626     -0.227      0.820      -1.373       1.089
CONF_B10      -8.9142      0.608    -14.667      0.000     -10.110      -7.719
CONF_B12     -10.4743      0.635    -16.491      0.000     -11.724      -9.225
CONF_BE       -7.1123      0.700    -10.162      0.000      -8.489      -5.735
CONF_BSky      2.7409      0.708      3.873      0.000       1.349       4.133
CONF_BSth      3.5642      0.752      4.737      0.000       2.084       5.044
CONF_BW        2.1300      0.663      3.211      0.001       0.825       3.435
CONF_CAA       3.1416      0.602      5.220      0.000       1.958       4.326
CONF_CUSA     -0.3227      0.692     -0.466      0.641      -1.684       1.038
CONF_Horz      3.5478      0.681      5.213      0.000       2.209       4.887
CONF_Ivy       0.1727      0.781      0.221      0.825      -1.364       1.710
CONF_MAAC      4.8949      0.627      7.812      0.000       3.662       6.127
CONF_MAC       3.6345      0.654      5.560      0.000       2.349       4.920
CONF_MEAC      6.5335      0.804      8.125      0.000       4.952       8.115
CONF_MVC       0.5186      0.644      0.805      0.421      -0.749       1.786
CONF_MWC      -4.2367      0.674     -6.283      0.000      -5.563      -2.910
CONF_NEC       6.7516      0.752      8.978      0.000       5.272       8.231
CONF_OVC       5.7880      0.672      8.612      0.000       4.466       7.110
CONF_Pat       5.4303      0.712      7.625      0.000       4.029       6.831
CONF_SB        2.7729      0.597      4.642      0.000       1.598       3.948
CONF_SC        4.0438      0.706      5.727      0.000       2.655       5.433
```

```
CONF_SEC     -10.1536      0.671    -15.122      0.000     -11.474      -8.833
CONF_SWAC      6.4211      0.672      9.556      0.000       5.099       7.743
CONF_Slnd      2.2868      0.637      3.590      0.000       1.034       3.540
CONF_Sum       1.4517      0.761      1.908      0.057      -0.045       2.949
CONF_WAC       0.8091      0.722      1.120      0.264      -0.612       2.230
CONF_WCC      -3.1289      0.681     -4.595      0.000      -4.468      -1.789
==============================================================================
Omnibus:                        2.794   Durbin-Watson:                   1.862
Prob(Omnibus):                  0.247   Jarque-Bera (JB):                3.012
Skew:                           0.033   Prob(JB):                        0.222
Kurtosis:                       3.441   Cond. No.                     2.43e+18
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The smallest eigenvalue is 1.4e-30. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
```

In [209…

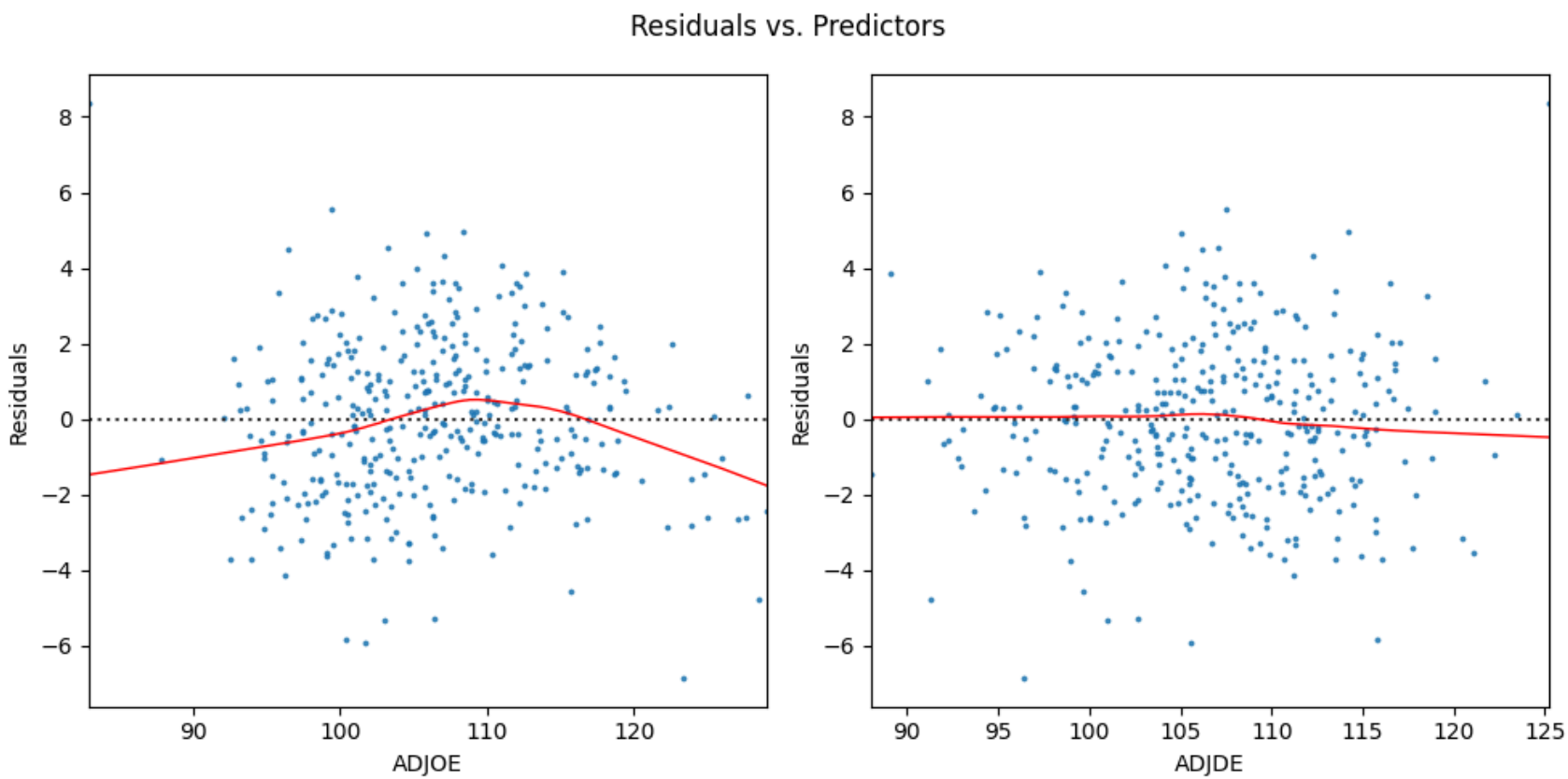
```python
# After fitting your model
basket['residuals'] = model.resid

# Then update your plot function to use basketball_dummies
def res_vs_pred(variable, ax):
    sns.residplot(x=basket[variable],
                  y=basket['residuals'],
                  lowess=True,
                  scatter_kws={'s': 3},
                  line_kws={'color': 'red', 'lw': 1},
                  ax=ax)
    ax.set_xlabel(variable)
    ax.set_ylabel("Residuals")
```

(L) Linear

In [210…

```python
# residual vs. predictor plots
def res_vs_pred(variable, ax):
    sns.residplot(x = basket[variable],
                  y = basket['residuals'],
                  lowess = True,
                  scatter_kws = {'s': 3},
                  line_kws = {'color': 'red', 'lw': 1},
```

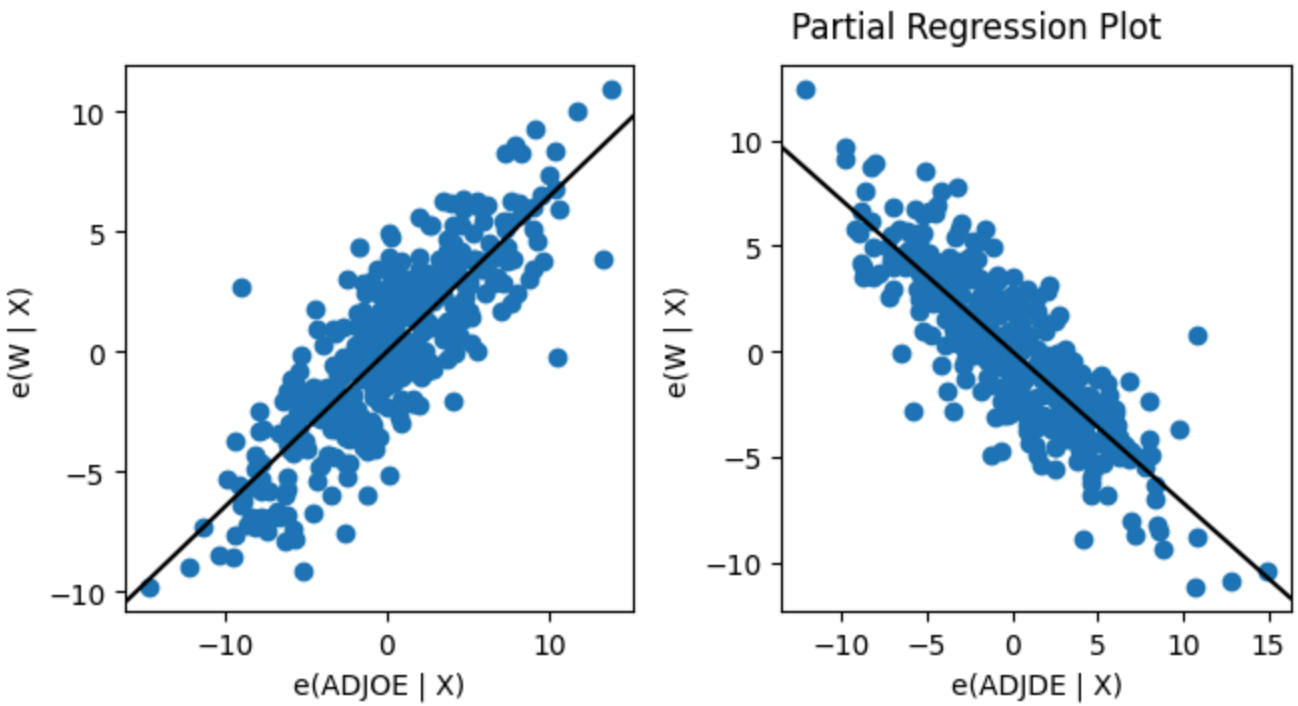
```
                    ax = ax)
    ax.set_xlabel(variable)
    ax.set_ylabel("Residuals")

fig, axes = plt.subplots(1, 2, figsize=(10, 5))
plt.suptitle("Residuals vs. Predictors")
res_vs_pred("ADJOE", axes[0])
res_vs_pred("ADJDE", axes[1])
fig.tight_layout()
plt.show()
```



In [211…

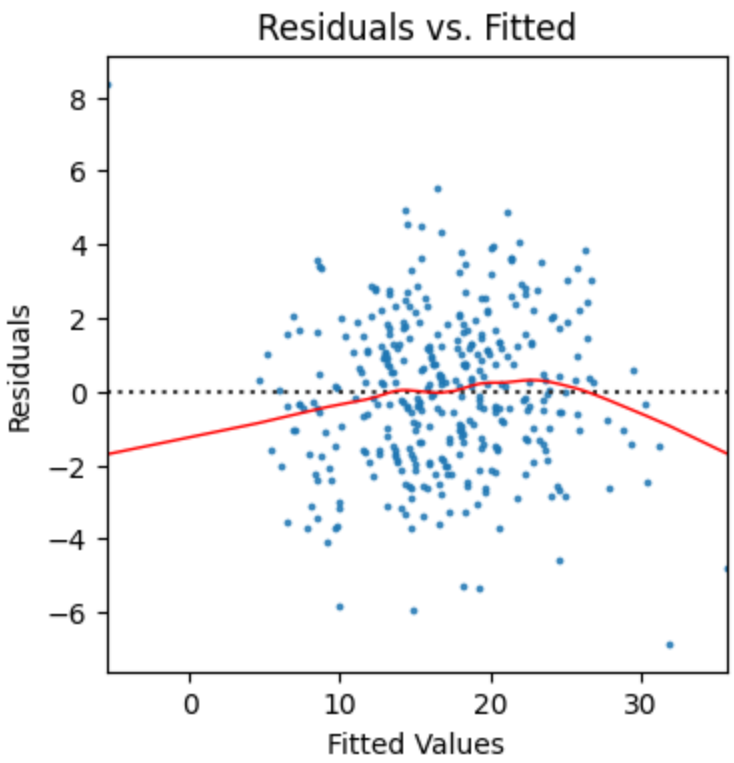
```
fig = plt.figure(figsize = (10,10))
sm.graphics.plot_partregress_grid(res,
                                  exog_idx = list(basket.columns[1:3]),
                                  grid = (3, 3),
                                  fig = fig)
```

```
fig.tight_layout()
plt.show()
```



In [212…

```
fig = plt.figure(figsize = (4, 4))
sns.residplot(x = basket['fittedvalues'],
              y = basket['residuals'],
              lowess = True,
              scatter_kws = {'s': 3},
              line_kws = {'color': 'red', 'lw': 1})
plt.title("Residuals vs. Fitted")
plt.ylabel("Residuals")
plt.xlabel("Fitted Values")
plt.show()
```



The assumptions seem to be met on ADJDE as it is fitted in the model around 0 ADJOE seems be met as well there is not noticeable curvature. and residuals vs. fitted plots the red line curves away from 0 a good amount. The partial Regression Plots are good because they have a direction in trend ADJOE having a positive trend and ADJDE having a negative trend.

(I) Independant

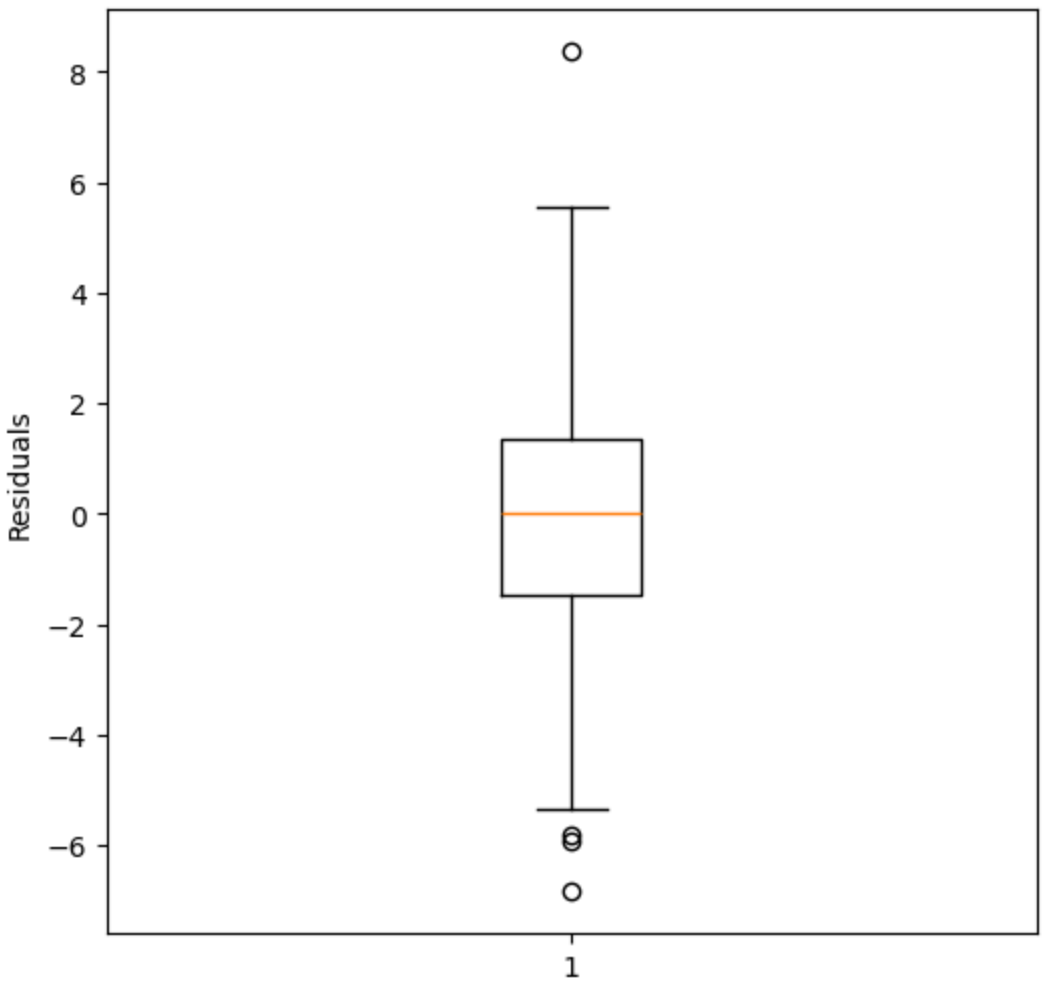
The assumption is met. Each observation is an individual team and none of them are repeated. We are also just looking at the overall data from just 2025 rather repeated yearly data.

(N) Normally distributed

In [213…

```
fig = plt.figure(figsize = (6, 6))
plt.boxplot(basket['residuals'])
```

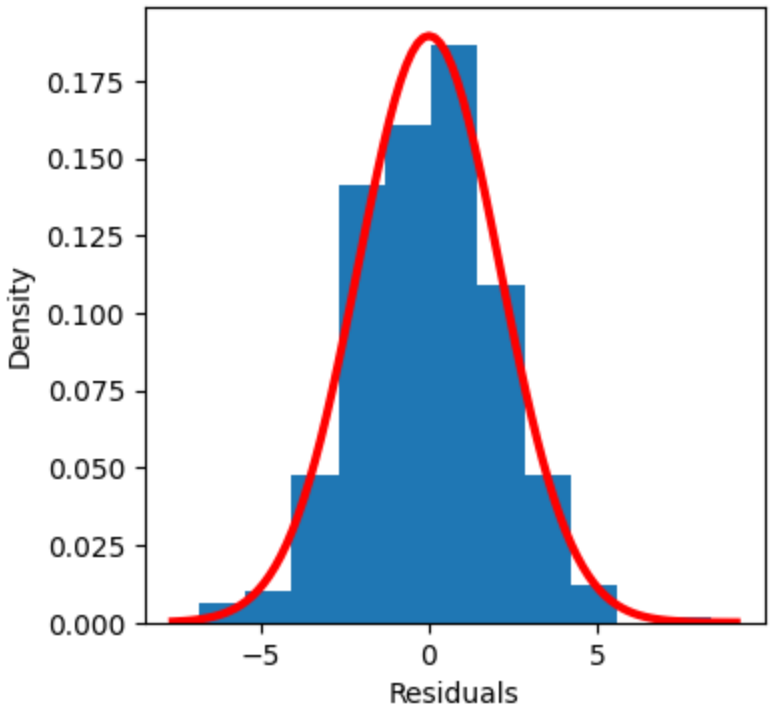
```
plt.ylabel("Residuals")
plt.show()
```



In [214…

```
# Diagnostic 2
fig = plt.figure(figsize = (4, 4))

# plot histogram (density = True so that it's on the same scale as the normal distribution)
plt.hist(basket['residuals'],
         density = True,
         bins = 11)
plt.xlabel("Residuals")
```

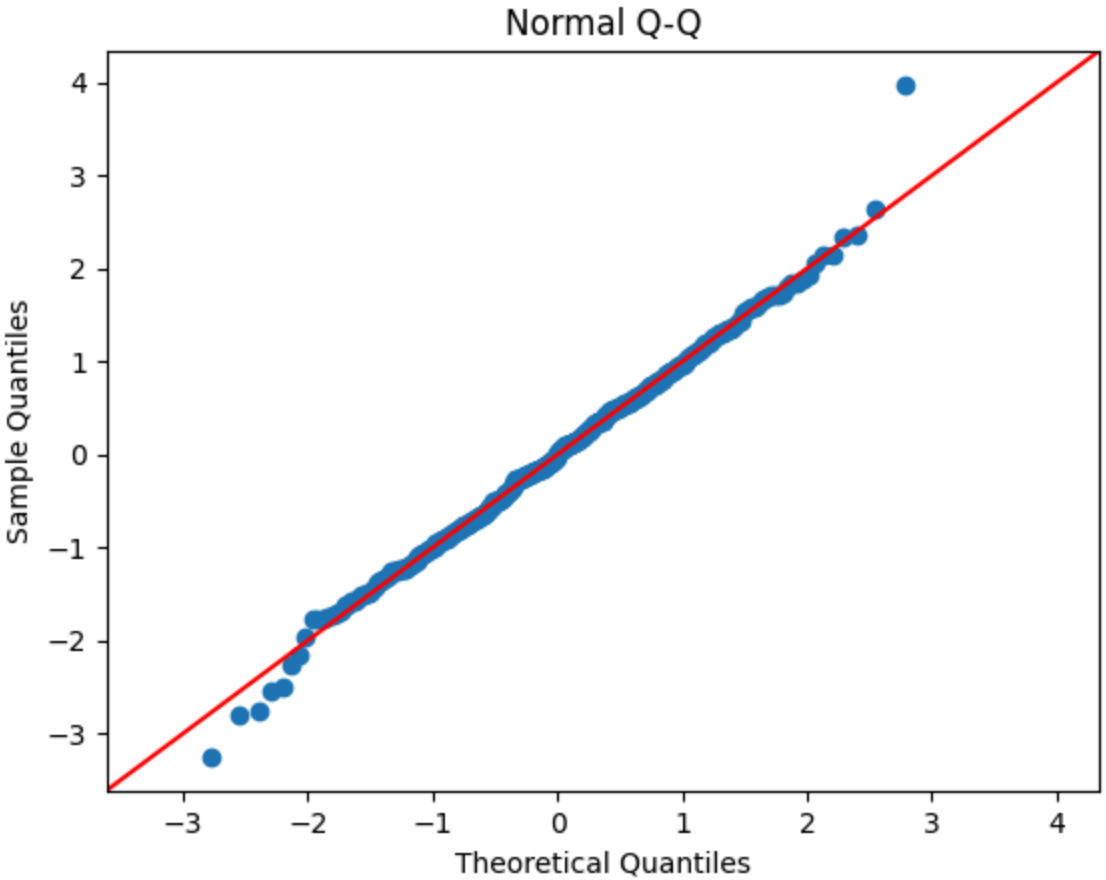
```
plt.ylabel("Density")

# calculate mean and standard deviation
mean = np.mean(basket['residuals'])
sd = np.std(basket['residuals'])

# generate x values to plot
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)

# plot normal distribution curve
plt.plot(x,
         stats.norm.pdf(x, mean, sd),
         color = 'r',
         lw = 3)
plt.show()
```



In [215…

```
# Diagnostic 3
sm.qqplot(basket['residuals'],
```

```
            line = '45',
            fit = True)
plt.title("Normal Q-Q")
plt.show()
```



In [216…

```
# Diagnostic 4
stats.shapiro(basket['residuals'])
```

Out[216…

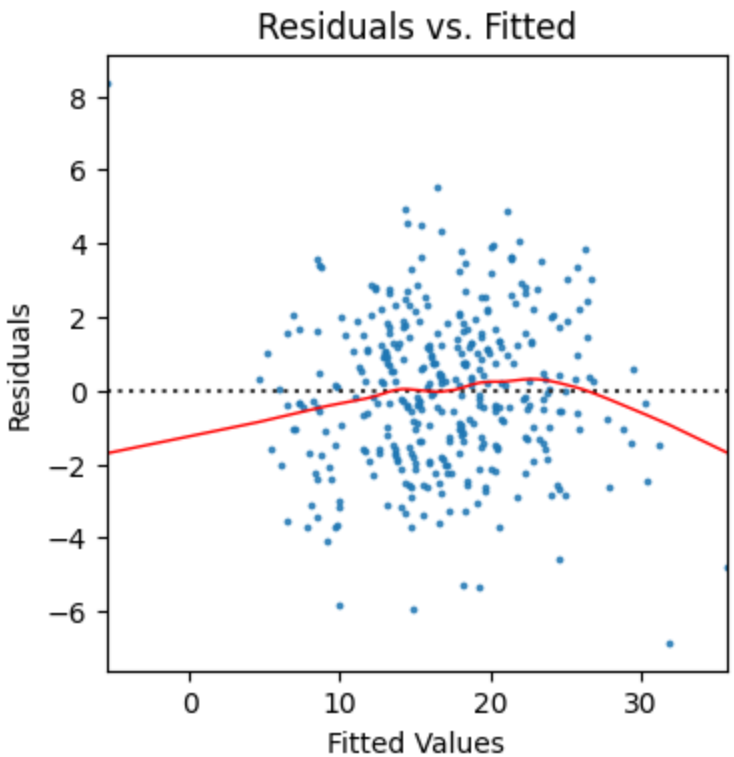
```
ShapiroResult(statistic=np.float64(0.9961631218823255), pvalue=np.float64(0.5279132922552059))
```

This assumption is met. There is possible tailing at the bottom of the QQ plot, some outliars in outside of the wicks in the box plot, the histogram seems to be slightly left skewed but the P-Value is higher than .05 so this is met. The P-Value is higher than the original model.

(E) Equal/Constant Variance

In [217…

```
fig = plt.figure(figsize = (4, 4))
sns.residplot(x = basket['fittedvalues'],
              y = basket['residuals'],
              lowess = True,
              scatter_kws = {'s': 3},
              line_kws = {'color': 'red', 'lw': 1})
plt.title("Residuals vs. Fitted")
plt.ylabel("Residuals")
plt.xlabel("Fitted Values")
plt.show()
```

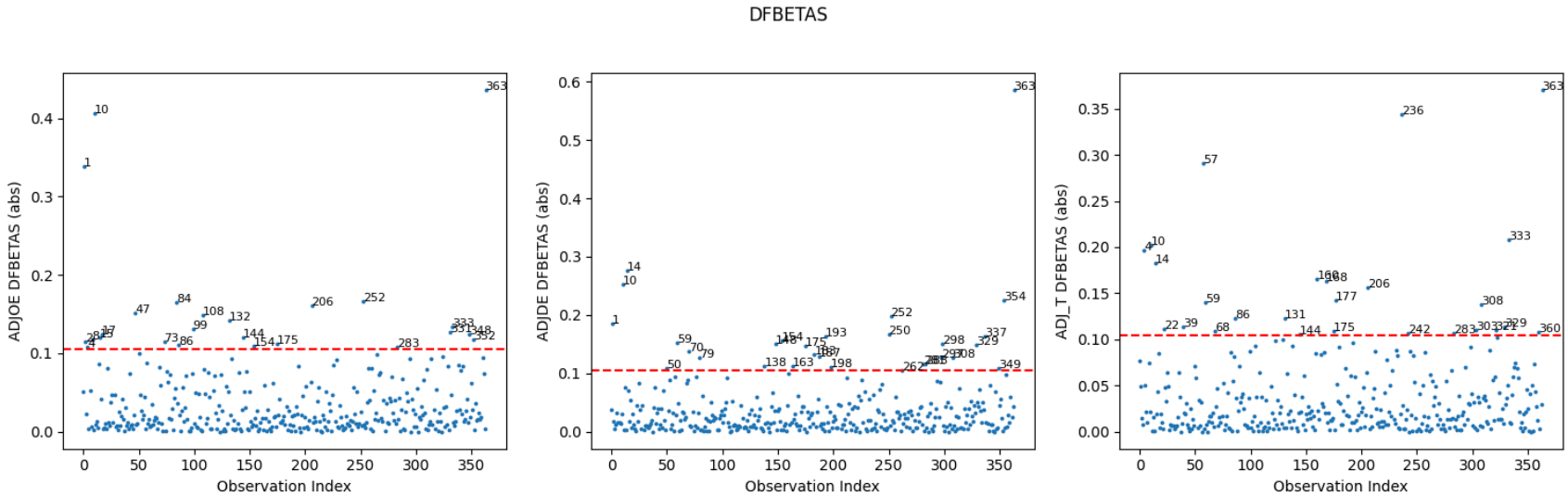


The assumption appears to be met. The vertical spread looks consistent throughout the plot.

(A) Describes all Observations

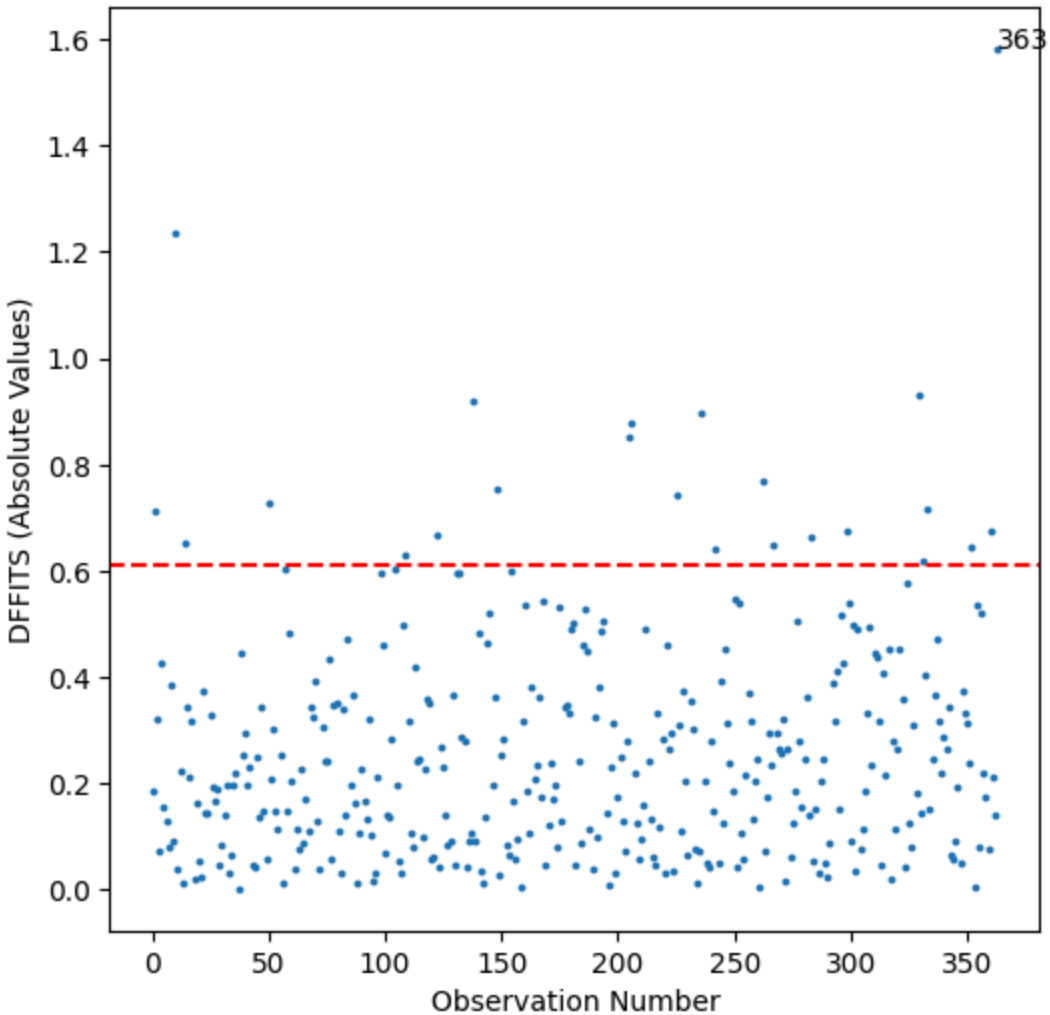
In [218…

```
# DFBETAS
def plot_dfbetas(column, ax, label_level=2 / np.sqrt(len(basket))):
    # Check if the column exists in the model's predictor names (exog_names)
    if column not in res.model.exog_names:
        print(f"Error: '{column}' was not used in the regression model.")
        return

    # Get the correct column index from the model results, not the original dataframe
    idx = res.model.exog_names.index(column)
    dfbetas = res.get_influence().dfbetas[:, idx]

    # Setup the plot
    ax.set_ylabel(f"{column} DFBETAS (abs)")
    ax.set_xlabel("Observation Index")
    ax.scatter(basket.index, np.abs(dfbetas), s=3)
    ax.axhline(y=label_level, color='r', linestyle='dashed', label='Threshold')

    # Label points exceeding the influence threshold
    outlier_indices = np.where(np.abs(dfbetas) > label_level)[0]
    for i in outlier_indices:
        # Use the actual index from your dataframe
        idx_label = basket.index[i]
        ax.annotate(str(idx_label), (idx_label, np.abs(dfbetas[i])), fontsize=8)

# Initialize the figure with the 1x3 layout
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
plt.suptitle("DFBETAS")

# Generate the three plots
plot_dfbetas("ADJOE", axes[0])
plot_dfbetas("ADJDE", axes[1])
plot_dfbetas("ADJ_T", axes[2])

fig.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



In [219…

```
# DFFITS
basket['dffits'] = res.get_influence().dffits[0]

fig, ax = plt.subplots(figsize = (6, 6))
ax.set_ylabel("DFFITS (Absolute Values)")
ax.set_xlabel("Observation Number")

# scatter + cutoff line
ax.scatter(basket.index,
           np.abs(basket['dffits']),
           s = 3)
ax.axhline(y = 2 * np.sqrt(len(res.params) / len(basket)),
           color = 'r',
           linestyle = 'dashed')

# Change the y_cutoff value to whatever number you want - all the
# points above that value will be labeled with their row number
y_cutoff_dffits = 1.3
outliers_dffits = basket[np.abs(basket['dffits']) > y_cutoff_dffits]
for i in outliers_dffits.index:
    ax.annotate(str(i), (i, np.abs(basket['dffits'][i])))

plt.show()
```



The assumption is met. In each plot, there are points that fall above the red line, all look good except for the plot 363 is questionable but after removing it from the model and checking all assumption not much changes so we assume that it is not an outliar.

In [220…

```
basket.iloc[363]
```

Out[220…

```
W                 3.000000
ADJOE            82.900000
ADJDE           125.200000
ADJ_T            64.800000
residuals         8.365132
fittedvalues     -5.525737
dffits            1.582684
CONF_A10          0.000000
CONF_ACC          0.000000
CONF_AE           0.000000
CONF_ASun         0.000000
CONF_Amer         0.000000
CONF_B10          0.000000
CONF_B12          0.000000
CONF_BE           0.000000
CONF_BSky         0.000000
CONF_BSth         0.000000
CONF_BW           0.000000
CONF_CAA          0.000000
CONF_CUSA         0.000000
CONF_Horz         0.000000
CONF_Ivy          0.000000
CONF_MAAC         0.000000
CONF_MAC          0.000000
CONF_MEAC         0.000000
CONF_MVC          0.000000
CONF_MWC          0.000000
CONF_NEC          0.000000
CONF_OVC          0.000000
CONF_Pat          0.000000
CONF_SB           0.000000
CONF_SC           0.000000
CONF_SEC          0.000000
CONF_SWAC         1.000000
CONF_Slnd         0.000000
CONF_Sum          0.000000
CONF_WAC          0.000000
CONF_WCC          0.000000
Name: 363, dtype: float64
```

We removed dot 363 to see if it had a significant change on the chart. We found that the assumptions aren't changed much so we decided to keep dot 363 in the model as normal.

In [221…

```
# 1. Filter the dataframe for rows where 'fittedvalues' is less than 0
negative_preds = basket[basket['fittedvalues'] < 0]

# 2. Count how many there are
print(f"Number of teams with negative predicted wins: {len(negative_preds)}")

# 3. View the data for these teams
print(negative_preds[['fittedvalues', 'residuals', 'W']])
```

```
Number of teams with negative predicted wins: 1
     fittedvalues  residuals  W
363     -5.525737   8.365132  3
```

This seemed like a huge outlier. It looks like this team won 3 games but they were actually predicted to win -5.5 games which is physically impossible. This is concerning in our dataset. the model is good at predicting all teams but this one. I have taken a look and this is the only team with negative wins predicted. overall though our model is good at predicting wins. But after removing 363 from the model and checking the assumptions we came to the conclusion that it was not an outliar.

(R) Additional predictor variables are not required

This assumption is reasonably met. We are taking into account several different predictor variables that we have found have a significant statistical correlation with the wins. Years of experiece the players have, different injuries that could have occured in the season. These would be nice to have in the model to see what significance they have.

No Multicolinearity

In [222…

```
basket_vifs = pd.DataFrame()
basket_vifs['Feature'] = X.columns[1:3]
basket_vifs['VIF'] = [vif(X, i) for i in range(1, 3)]

print("Max = ", max(basket_vifs['VIF']))
print("Mean = ", np.mean(basket_vifs['VIF']))
basket_vifs
```

```
Max =  2.4372320248053785
Mean =  2.3460038557404523
```

Out[222…

	Feature	VIF
0	ADJOE	2.437232
1	ADJDE	2.254776

This assumption is not violated. The VIF for all are around 2 which is pretty good. There doesn't really seem to be multicollinearity. They all seem to give something very different in the data. and that each varible gives us something different and independant from the other varibles.

Overall, based on our diagnostics our assumptions were satisfied. Normal distribution seems to be met. And after Removing and checking the DFBETAS and DFFITS again we see that 363 doesn't effect the model significantly, so we have decided to keep it in.

In [223…

```
print(basket.columns.tolist())
```

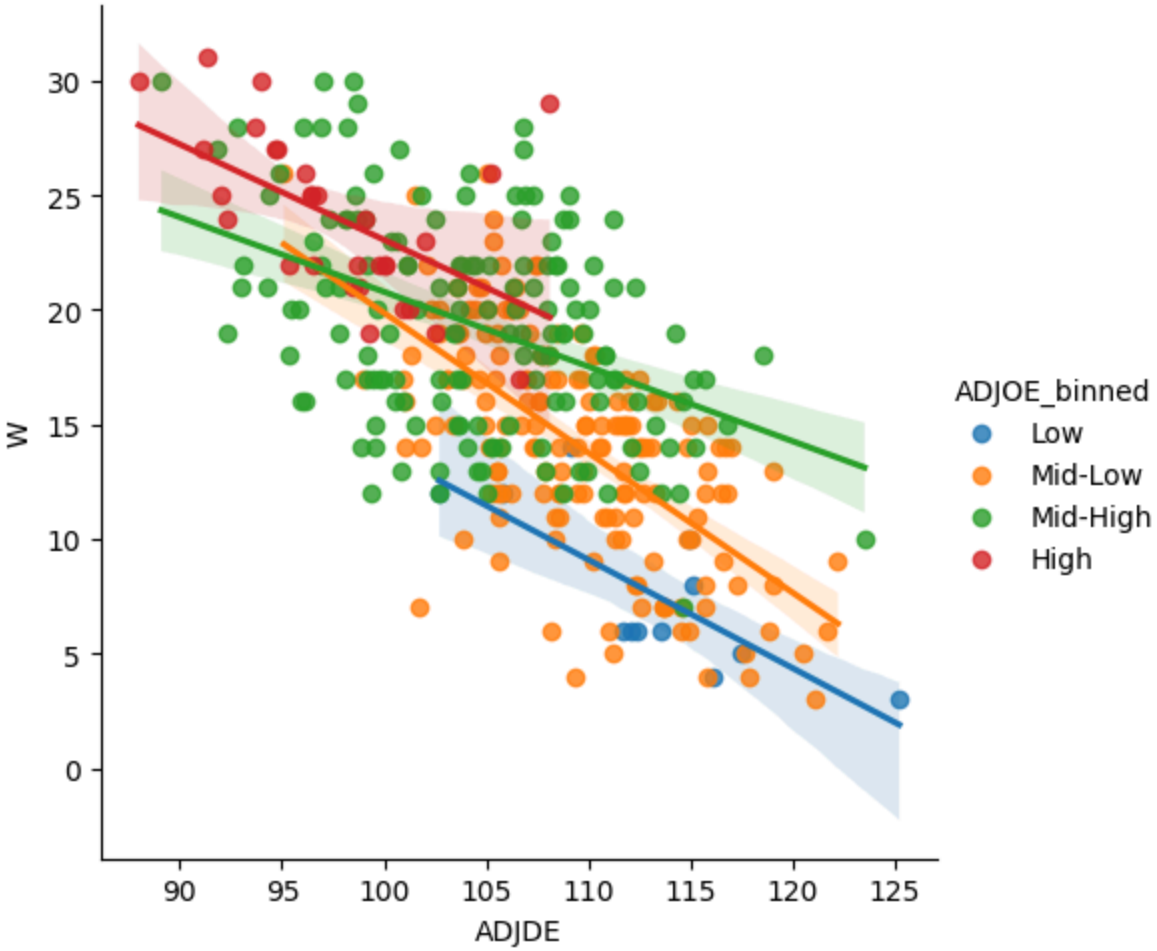
```
['W', 'ADJOE', 'ADJDE', 'ADJ_T', 'residuals', 'fittedvalues', 'dffits', 'CONF_A10', 'CONF_ACC', 'CONF_AE',
'CONF_ASun', 'CONF_Amer', 'CONF_B10', 'CONF_B12', 'CONF_BE', 'CONF_BSky', 'CONF_BSth', 'CONF_BW', 'CONF_CA
A', 'CONF_CUSA', 'CONF_Horz', 'CONF_Ivy', 'CONF_MAAC', 'CONF_MAC', 'CONF_MEAC', 'CONF_MVC', 'CONF_MWC', 'CO
NF_NEC', 'CONF_OVC', 'CONF_Pat', 'CONF_SB', 'CONF_SC', 'CONF_SEC', 'CONF_SWAC', 'CONF_Slnd', 'CONF_Sum', 'C
ONF_WAC', 'CONF_WCC']
```

In [224…

```
# First, EDA to see if there is any indicate this interaction exists:
# Create 4 bins for the hue variable
basket['ADJOE_binned'] = pd.cut(basket['ADJOE'], bins=4, labels=['Low', 'Mid-Low', 'Mid-High', 'High'])

sns.lmplot(data=basket,
           x="ADJDE",
           y="W",
           hue="ADJOE_binned",
           height=5,
           aspect=1)
```

Out[224…

```
<seaborn.axisgrid.FacetGrid at 0x12e17f890>
```

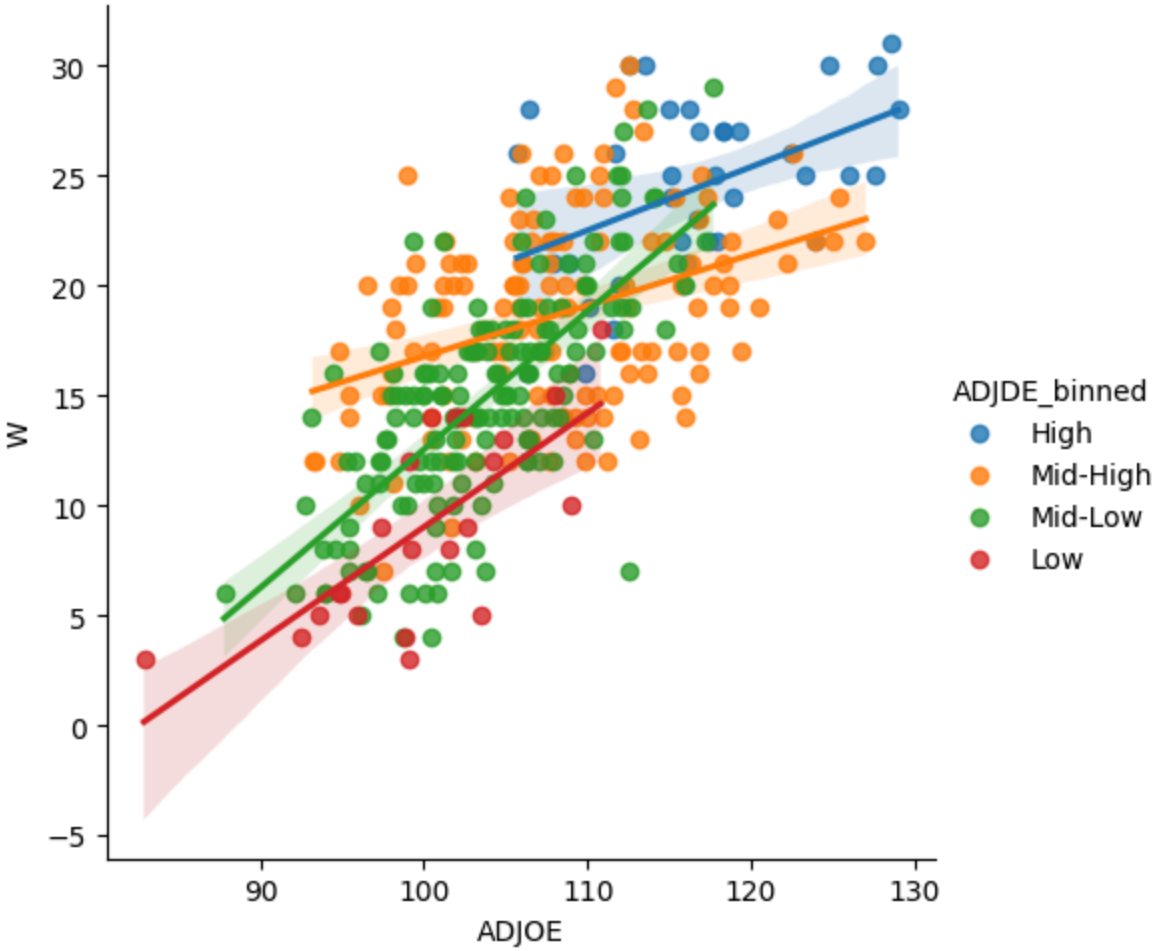


In [225…

```
basket['ADJDE_binned'] = pd.cut(basket['ADJDE'], bins=4, labels=['High', 'Mid-High', 'Mid-Low', 'Low'])

sns.lmplot(data=basket,
           x="ADJOE",
           y="W",
           hue="ADJDE_binned",
           height=5,
           aspect=1)
```

Out[225…

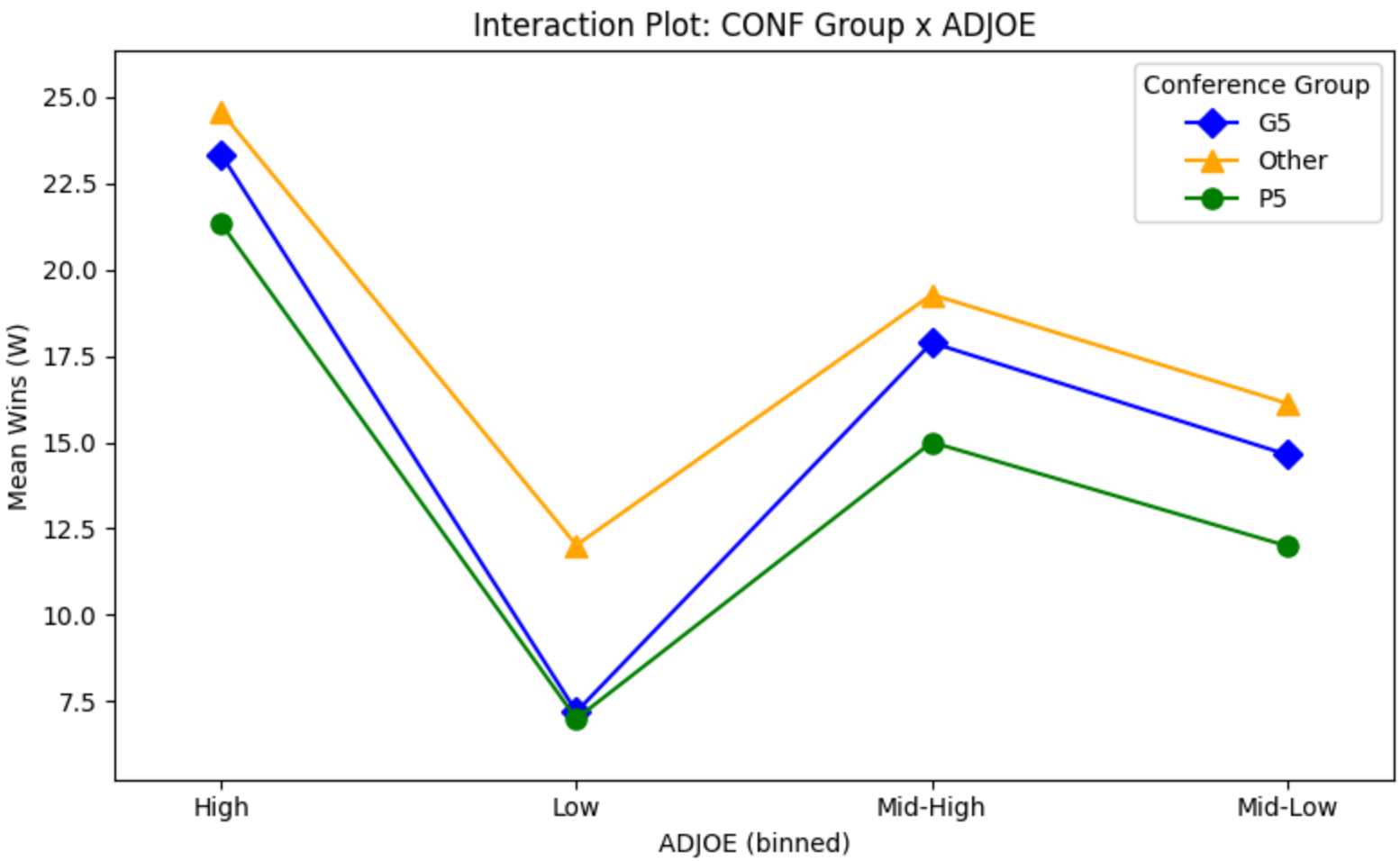
```
<seaborn.axisgrid.FacetGrid at 0x12cebc7d0>
```



In [226…

```
from statsmodels.graphics.factorplots import interaction_plot

# Step 1: Bin ADJOE and convert to string
basket['ADJOE_binned'] = pd.qcut(basket['ADJOE'], q=4, labels=['Low', 'Mid-Low', 'Mid-High', 'High']).asty

# Step 2: Define P5/G5 dummy columns
P5_cols = ['CONF_ACC', 'CONF_B10', 'CONF_B12', 'CONF_SEC', 'CONF_BE']
G5_cols = ['CONF_A10', 'CONF_MWC', 'CONF_WCC', 'CONF_MVC', 'CONF_Amer']

# Step 3: Create group column from dummies
basket['CONF_group'] = 'Other'
basket.loc[basket[P5_cols].sum(axis=1) == 1, 'CONF_group'] = 'P5'
```

```
basket.loc[basket[G5_cols].sum(axis=1) == 1, 'CONF_group'] = 'G5'

# Step 4: Interaction plot
fig, ax = plt.subplots(figsize=(8, 5))

interaction_plot(
    x=basket['ADJOE_binned'],
    trace=basket['CONF_group'],
    response=basket['W'],
    colors=['blue', 'orange', 'green'],
    markers=['D', '^', 'o'],
    ms=8,
    ax=ax
)

ax.set_title('Interaction Plot: CONF Group x ADJOE')
ax.set_xlabel('ADJOE (binned)')
ax.set_ylabel('Mean Wins (W)')
ax.legend(title='Conference Group')
plt.tight_layout()
plt.show()
```



In [227…

```
basket['ADJDE_x_ADJOE'] = basket['ADJDE'] * basket['ADJOE']
# basket.head()
X = basket[features_base + features_conf + ['ADJDE_x_ADJOE']]
y = basket['W']

X = sm.add_constant(X)
model = sm.OLS(y, X).fit()

print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      W   R-squared:                       0.873
Model:                            OLS   Adj. R-squared:                  0.860
Method:                 Least Squares   F-statistic:                     68.44
Date:                Wed, 25 Mar 2026   Prob (F-statistic):          2.98e-127
Time:                        12:22:53   Log-Likelihood:                -785.48
No. Observations:                 364   AIC:                             1639.
Df Residuals:                     330   BIC:                             1771.
Df Model:                          33
Covariance Type:            nonrobust
==============================================================================
                    coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const            71.6348     25.725      2.785      0.006      21.030     122.240
ADJOE             0.1839      0.248      0.741      0.459      -0.304       0.672
ADJDE            -1.1819      0.250     -4.725      0.000      -1.674      -0.690
CONF_A10          0.2154      0.967      0.223      0.824      -1.687       2.118
CONF_ACC         -4.6195      1.020     -4.531      0.000      -6.625      -2.614
CONF_AE           7.5122      1.115      6.737      0.000       5.319       9.706
CONF_ASun         4.7301      1.048      4.512      0.000       2.668       6.792
CONF_Amer         1.2734      0.977      1.303      0.193      -0.649       3.196
CONF_B10         -7.1953      1.096     -6.564      0.000      -9.352      -5.039
CONF_B12         -8.6678      1.150     -7.538      0.000     -10.930      -6.406
CONF_BE          -5.5228      1.095     -5.041      0.000      -7.678      -3.368
CONF_BSky         4.2045      1.050      4.005      0.000       2.139       6.270
CONF_BSth         5.0287      1.081      4.653      0.000       2.903       7.155
CONF_BW           3.6167      1.030      3.511      0.001       1.590       5.643
CONF_CAA          4.5904      0.976      4.703      0.000       2.670       6.510
CONF_CUSA         1.1016      1.024      1.076      0.283      -0.913       3.116
CONF_Horz         4.9969      1.026      4.869      0.000       2.978       7.016
CONF_Ivy          1.5999      1.087      1.472      0.142      -0.538       3.738
CONF_MAAC         6.4154      1.021      6.282      0.000       4.407       8.424
CONF_MAC          5.1188      1.023      5.004      0.000       3.106       7.131
CONF_MEAC         8.2097      1.198      6.852      0.000       5.853      10.567
CONF_MVC          1.9412      0.992      1.957      0.051      -0.010       3.892
CONF_MWC         -2.7035      1.056     -2.560      0.011      -4.781      -0.626
CONF_NEC          8.3762      1.143      7.327      0.000       6.127      10.625
CONF_OVC          7.3323      1.059      6.922      0.000       5.248       9.416
CONF_Pat          6.9983      1.095      6.394      0.000       4.845       9.152
CONF_SB           4.2585      0.989      4.307      0.000       2.313       6.204
CONF_SC           5.5532      1.067      5.204      0.000       3.454       7.652
```

```
CONF_SEC         -8.2511      1.213     -6.805      0.000     -10.636      -5.866
CONF_SWAC         8.2373      1.175      7.012      0.000       5.926      10.548
CONF_Slnd         3.7864      1.019      3.716      0.000       1.782       5.791
CONF_Sum          2.8197      1.050      2.684      0.008       0.753       4.886
CONF_WAC          2.2384      1.047      2.139      0.033       0.180       4.297
CONF_WCC         -1.5604      1.075     -1.452      0.148      -3.675       0.554
ADJDE_x_ADJOE     0.0044      0.002      1.881      0.061      -0.000       0.009
==============================================================================
Omnibus:                       11.561   Durbin-Watson:                   1.903
Prob(Omnibus):                  0.003   Jarque-Bera (JB):               18.602
Skew:                           0.198   Prob(JB):                     9.13e-05
Kurtosis:                       4.034   Cond. No.                     1.80e+20
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The smallest eigenvalue is 1.44e-30. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
```

After fitting a model with ADJOE and ADJDE interactions The P value is not statistically significant so we will just be using the model with no interactions the rest of the way.

In [234…

```python
from sklearn.metrics import mean_squared_error, mean_absolute_error

fitted    = model.fittedvalues
residuals = model.resid

# Model Assessment
mse    = mean_squared_error(y, fitted)
rmse   = np.sqrt(mse)
mae    = mean_absolute_error(y, fitted)
r2     = model.rsquared
adj_r2 = model.rsquared_adj
aic    = model.aic
bic    = model.bic
f_stat = model.fvalue
f_pval = model.f_pvalue

print(f'MSE:          {mse:.4f}')
print(f'RMSE:         {rmse:.4f}')
print(f'MAE:          {mae:.4f}')
```

```
print(f'R²:           {r2:.4f}')
print(f'Adjusted R²:  {adj_r2:.4f}')
print(f'AIC:          {aic:.4f}')
print(f'BIC:          {bic:.4f}')
print(f'F-statistic:  {f_stat:.4f}')
print(f'F p-value:    {f_pval:.4f}')
```

```
MSE:          4.3839
RMSE:         2.0938
MAE:          1.6493
R²:           0.8725
Adjusted R²:  0.8598
AIC:          1638.9569
BIC:          1771.4601
F-statistic:  68.4365
F p-value:    0.0000
```

Since 2.09 (RMSE) is higher than 1.65 (MAE), it tells us we have some "outliers"—teams that significantly over-performed or under-performed their stats (e.g., a team that "should" have won 20 games based on efficiency but only won 14 due to injuries or bad luck).

R^2 (0.8725): Our model explains 87.25% of the variance in wins. This is a very high "signal-to-noise" ratio.Adjusted R^2 (0.8598): This is the "honest" version of R^2. It penalizes the model for adding variables (like those 33 conferences) that don't add enough value. The fact that this is slightly lower than the regular R^2 means some of our variables might be "dead weight."

AIC (1638.96) & BIC (1771.46): These numbers don't mean much alone, but they represent the trade-off between accuracy and complexity. Lower is better. If we remove the interaction term and the BIC drops to 1750, the simpler model is mathematically superior. The high BIC here reflects that our model has a lot of "moving parts" (33 variables).

F-statistic (68.44) & F p-value (0.0000): This is our "Confidence Score." The p-value of 0.0000 means the probability that these results happened by random chance is effectively zero. Our model is highly statistically significant.

Statistical Inference

In [229…

```
# coefficient table
coef_table = pd.DataFrame({
    'coef': res.params,
    'p_value': res.pvalues
})
```

```python
# 95% confidence intervals
ci_table = res.conf_int()
ci_table.columns = ['lower_95', 'upper_95']

# combine everything
infer_table = pd.concat([coef_table, ci_table], axis=1)

# remove intercept if present
if 'const' in infer_table.index:
    infer_table = infer_table.drop(index='const')

# only statistically significant slopes
sig_slopes = infer_table[infer_table['p_value'] < 0.05]

sig_slopes
```

Out[229…

	coef	p_value	lower_95	upper_95
ADJOE	0.646501	1.686021e-86	0.599844	0.693158
ADJDE	-0.717446	1.659461e-85	-0.769744	-0.665147
CONF_ACC	-4.962176	6.670372e-10	-6.496664	-3.427688
CONF_AE	7.187555	6.335555e-13	5.299933	9.075178
CONF_ASun	4.457379	7.001336e-07	2.724224	6.190534
CONF_B10	-7.701865	1.758391e-19	-9.276395	-6.127336
CONF_B12	-9.252310	6.506606e-25	-10.876608	-7.628013
CONF_BE	-5.860146	1.580281e-10	-7.605152	-4.115140
CONF_BSky	4.014044	1.594806e-05	2.211292	5.816797
CONF_BSth	4.781095	7.807141e-07	2.913792	6.648398
CONF_BW	3.347611	1.762879e-04	1.611899	5.083323
CONF_CAA	4.415527	2.214093e-07	2.774056	6.056999
CONF_Horz	4.799636	1.515191e-07	3.040254	6.559017
CONF_MAAC	6.164405	6.576712e-12	4.462445	7.866365
CONF_MAC	4.829602	6.989552e-08	3.107434	6.551770
CONF_MEAC	7.737920	1.954338e-13	5.753234	9.722606
CONF_MVC	1.791012	3.781035e-02	0.101450	3.480573
CONF_MWC	-2.981119	7.675948e-04	-4.708030	-1.254207
CONF_NEC	8.048301	3.370393e-15	6.133444	9.963157
CONF_OVC	7.034758	9.827655e-14	5.254436	8.815079
CONF_Pat	6.724639	3.797343e-12	4.890380	8.558898
CONF_SB	4.036602	1.967465e-06	2.397050	5.676154
CONF_SC	5.301418	1.671118e-08	3.499138	7.103698

	coef	p_value	lower_95	upper_95
CONF_SEC	-8.965315	7.250900e-23	-10.625619	-7.305012
CONF_SWAC	7.626046	1.826335e-15	5.830742	9.421350
CONF_SInd	3.548914	5.372844e-05	1.842974	5.254854
CONF_Sum	2.679509	5.018807e-03	0.813512	4.545506
CONF_WAC	2.013284	3.319643e-02	0.161435	3.865134
CONF_WCC	-1.909653	3.064247e-02	-3.640009	-0.179296

Confidence Intervals:

We are 95% confident that the true effect of ADJOE on wins is between 0.600 and 0.693, holding defensive efficiency and conference constant. This means that a one-unit increase in adjusted offensive efficiency is associated with an increase of approximately 0.60 to 0.69 wins on average.

We are 95% confident that the true effect of ADJDE on wins is between -0.770 and -0.665, holding offensive efficiency and conference constant. This indicates that a one-unit increase in adjusted defensive efficiency is associated with a decrease of approximately 0.67 to 0.77 wins on average.

Hypothesis test:

Since the p-values for the predictors are less than 0.05, we reject the null hypothesis for these variables. This provides sufficient evidence that adjusted offensive efficiency (ADJOE), adjusted defensive efficiency (ADJDE), and conference affiliation have a statistically significant relationship with the number of wins in the population, holding all other variables constant.

In [236…

```
# get the exact predictor names used in the fitted model
model_cols = res.model.exog_names

# create one row of zeros with those exact columns
example = pd.DataFrame(np.zeros((1, len(model_cols))), columns=model_cols)

# set intercept
if 'const' in example.columns:
```

```
    example['const'] = 1

# set continuous predictors
if 'ADJOE' in example.columns:
    example['ADJOE'] = basket['ADJOE'].mean()

if 'ADJDE' in example.columns:
    example['ADJDE'] = basket['ADJDE'].mean()

# choose one conference
if 'CONF_SEC' in example.columns:
    example['CONF_SEC'] = 1

# get confidence interval for mean response
mean_ci = res.get_prediction(example).summary_frame(alpha=0.05)

mean_ci[['mean', 'mean_ci_lower', 'mean_ci_upper']]
```

Out[236…

	mean	mean_ci_lower	mean_ci_upper
0	3.706276	-3.098216	10.510768

Confidence Intervals:

We are 95% confident that the true effect of ADJOE on wins is between 0.600 and 0.693, holding defensive efficiency and conference constant. This means that a one-unit increase in adjusted offensive efficiency is associated with an increase of approximately 0.60 to 0.69 wins on average.

We are 95% confident that the true effect of ADJDE on wins is between -0.770 and -0.665, holding offensive efficiency and conference constant. This indicates that a one-unit increase in adjusted defensive efficiency is associated with a decrease of approximately 0.67 to 0.77 wins on average.

Hypothesis test:

Since the p-values for the predictors are less than 0.05, we reject the null hypothesis for these variables. This provides sufficient evidence that adjusted offensive efficiency (ADJOE), adjusted defensive efficiency (ADJDE), and conference affiliation have a statistically significant relationship with the number of wins in the population, holding all other variables constant.

In [235…

```
# get the exact predictor names used in the fitted model
model_cols = res.model.exog_names

# create one row of zeros with those exact columns
example = pd.DataFrame(np.zeros((1, len(model_cols))), columns=model_cols)

# set intercept
if 'const' in example.columns:
    example['const'] = 1

# set continuous predictors
if 'ADJOE' in example.columns:
    example['ADJOE'] = basket['ADJOE'].mean()

if 'ADJDE' in example.columns:
    example['ADJDE'] = basket['ADJDE'].mean()

# choose one conference
if 'CONF_SEC' in example.columns:
    example['CONF_SEC'] = 1

# get confidence interval for mean response
mean_ci = res.get_prediction(example).summary_frame(alpha=0.05)

mean_ci[['mean', 'mean_ci_lower', 'mean_ci_upper']]
```

Out[235…

	mean	mean_ci_lower	mean_ci_upper
0	3.706276	-3.098216	10.510768

Confidence interval for mean of Y:

We are 95% confident that the average number of wins for teams with average ADJOE, average ADJDE, and in the SEC lies between -3.10 and 10.51 wins. This interval is for the mean response, so it describes the average wins for all teams with those predictor values, not the wins of a single team.

In [231…

```
pred_ci = res.get_prediction(example).summary_frame(alpha=0.05)

pred_ci[['mean', 'obs_ci_lower', 'obs_ci_upper']]
```

Out[231…

	mean	obs_ci_lower	obs_ci_upper
0	3.706276	-4.366488	11.77904

Prediction interval:

We are 95% confident that the number of wins for a new team with average ADJOE, average ADJDE, and in the SEC will fall between -4.37 and 11.78 wins. This interval is wider than the confidence interval for the mean because it accounts for both uncertainty in estimating the mean and the natural variability in wins for individual teams. It's also worth noting that the lower end is negative which does reflect the variability but isn't actually possible for wins.

The purpose of this analysis was to see if ADJOE, ADJDE and ADJ_T could be used to predict the amount of wins a college basketball team gets in a season. The results show that performance metrics and which conference the team is apart of play a significant role in predicting wins.

We found that ADJOE has a strong, positive, linear relationship with wins while ADJDE has a strong, negative, linear relationship with wins. Also, we found that conference has an impact on a teams success. Some conferences were associated with higher expected wins, while others were associated with lower expected wins. We also found that ADJ_T did not add to the value of this analysis so we removed it from the model.

Overall, we found that conference affiliation and efficiency metrics help understand team performance. These results could be useful for coaches, analysts, or organizations.

In [ ]: